

SRTT por una constante [Ecuación (17.4)], se suma a SRTT un múltiplo de la desviación media estimada para formar el temporizador de retransmisión. Basándose en sus experimentos de temporización, Jacobson propone en su artículo original los siguientes valores para las constantes:

$$\begin{aligned}g &= 1/8 = 0,125 \\h &= 1/4 = 0,25 \\f &= 2\end{aligned}$$

Después de investigaciones posteriores [JACO90], recomendó cambiar el valor de f a 4, y éste es el valor estándar utilizado en la implementación actual.

La Figura 17.13 muestra el uso de la Ecuación (17.5) en el mismo conjunto de datos que se utilizó en la Figura 17.12. Una vez que el tiempo de llegada se estabiliza, la estimación de la variación SDEV declina. El valor de RTO para ambos casos $f = 2$ y $f = 4$ es bastante conservador mientras RTT está cambiando, pero comienza a converger a RTT cuando se estabiliza.

La experiencia ha demostrado que el algoritmo de Jacobson puede mejorar significativamente el rendimiento de TCP. Sin embargo, no se basta por sí mismo. Se deben considerar otros dos factores:

1. ¿Qué valor de RTO se debería utilizar en un segmento retransmitido? Para este propósito se utiliza el algoritmo de decaimiento exponencial de RTO.
2. ¿Qué muestras se deberían utilizar como entrada al algoritmo de Jacobson? El algoritmo de Karn determina qué muestras se utilizan.

Decaimiento exponencial de RTO

Cuando expira un temporizador en el emisor TCP, debe retransmitir ese segmento. El RFC 793 supone que se va a utilizar el mismo RTO para este segmento retransmitido. Sin embargo, ya que el que expire un temporizador se debe probablemente a la congestión de red, manifestada en el descarte del paquete o en un retardo grande en el tiempo de ida y vuelta, no es aconsejable mantener el mismo valor de RTO.

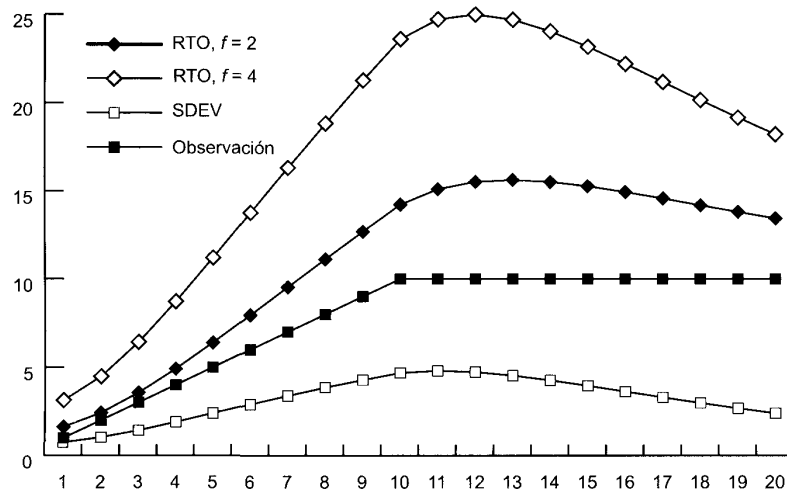
Considere el siguiente escenario. Existen varias conexiones TCP activas de varias fuentes enviando tráfico a un conjunto de redes. Aparece la congestión en una región de forma que los segmentos en muchas de las conexiones se pierden o retrasan más allá del tiempo RTO de las conexiones. Por lo tanto, casi al mismo tiempo, muchos segmentos serán retransmitidos hacia el conjunto de redes, manteniendo o incluso incrementando la congestión. Todas las fuentes esperan un tiempo RTO local (para cada conexión) y retransmiten de nuevo. Este modelo de comportamiento podría causar una condición de congestión continua.

Un criterio más sensible dicta que la fuente TCP incremente su RTO cada vez que se retransmite un segmento; esto se conoce como proceso de *decaimiento*. En el escenario de párrafo anterior, después de la primera retransmisión de un segmento en cada conexión afectada las fuentes TCP esperarán un tiempo mayor antes de intentar la segunda retransmisión. Esto le da tiempo al conjunto de redes a despejar la congestión actual. Si se necesita una segunda retransmisión, cada fuente TCP esperará un tiempo mayor todavía antes de que expire el temporizador para una tercera retransmisión, dando al conjunto un periodo mayor todavía para recuperarse.

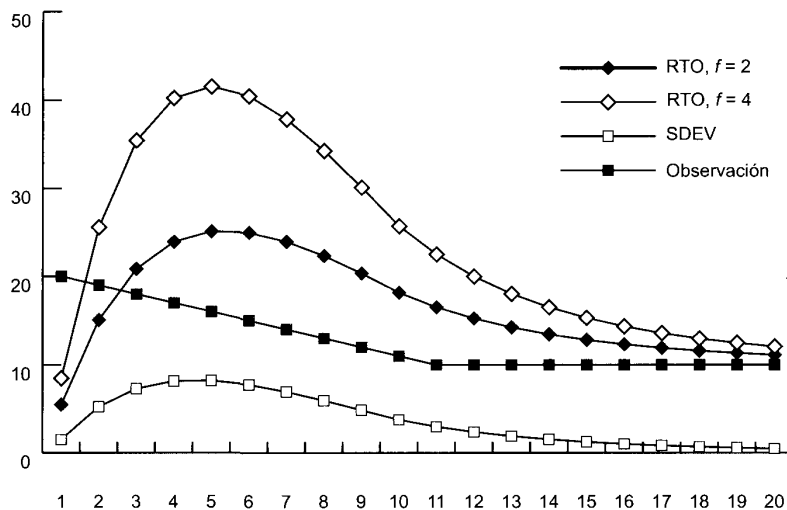
Una técnica simple para implementar el decaimiento de RTO es multiplicar el RTO de un segmento por una valor constante para cada retransmisión:

$$RTO = q \times RTO \quad (17.6)$$

La Ecuación (17.6) hace que el RTO crezca exponencialmente con cada retransmisión. El valor de q más utilizado comúnmente es 2. Con este valor, la técnica se conoce como *decaimiento exponencial binario*. Ésta es la misma técnica que se utiliza en el protocolo CSMA/CD de Ethernet (Capítulo 14).



(a) Función creciente



(b) Función decreciente

Figura 17.13. Cálculo del RTO de Jacobson.

Algoritmo de Karn

Si no se retransmiten segmentos, el proceso de muestreo del algoritmo de Jacobson es directo. El RTT de cada segmento se puede incluir en los cálculos. Suponga, sin embargo, que expira el temporizador de un segmento y se debe retransmitir. Si se recibe una confirmación, existen dos posibilidades:

1. Ésta es el ACK de la primera transmisión del segmento. En este caso, el RTT es simplemente mayor que el esperado pero es un reflejo preciso de las condiciones de la red.

2. Ésta es el ACK de la segunda transmisión.

La fuente TCP no puede distinguir entre estos dos casos. Si es verdadero el segundo caso y la entidad TCP mide simplemente el RTT desde la primera transmisión hasta la recepción del ACK, el tiempo medido será demasiado grande. El RTT medido será del orden del RTT actual más el RTO. Pasando este falso RTT al algoritmo de Jacobson producirán un valor alto innecesario de SRTT y por lo tanto de RTO. Además, este efecto se propaga hacia adelante un determinado número de iteraciones, ya que el valor de SRTT de una iteración es el valor de entrada en la siguiente iteración.

Un criterio incluso peor sería medir el RTT desde la segunda transmisión hasta la recepción del ACK. Si éste es en realidad el ACK de la primera transmisión, entonces el RTT medido sería demasiado pequeño, produciendo un valor demasiado pequeño de SRTT y RTO. Esto probablemente tenga un efecto de realimentación positiva, causando retransmisiones adicionales y medidas falsas adicionales.

El algoritmo de Karn [KARN91] resuelve este problema con las reglas siguientes:

1. No utilizar el RTT medido en una retransmisión para actualizar SRTT y SDEV [Ecuación (17.5)].
2. Calcular el RTO de decaimiento utilizando la Ecuación (17.6) cuando ocurra una retransmisión.
3. Utilizar el valor del RTO de decaimiento para sucesivos segmentos hasta que llegue una confirmación para un segmento que no se ha retransmitido.

Cuando se recibe una confirmación para un segmento que no se ha retransmitido, se activa de nuevo el algoritmo de Jacobson para calcular valores futuros de RTO.

GESTIÓN DE LA VENTANA

Además de las técnicas para mejorar la efectividad del temporizador de retransmisión, se han examinado una serie de criterios para gestionar la ventana de emisión. El tamaño de la ventana de emisión de TCP puede tener un efecto crítico en si TCP puede ser utilizado eficientemente sin causar congestión. Se discuten dos técnicas que se encuentran en virtualmente todas las implementaciones recientes de TCP: el comienzo lento y el ajuste dinámico de la ventana en caso de congestión.

Comienzo lento

Cuanto mayor es la ventana de emisión, más segmentos puede enviar la fuente TCP antes de que deba esperar una confirmación. Esto puede crear un problema cuando se establece por primera vez una conexión TCP, ya que la entidad TCP es libre de vaciar la ventana de datos completa en el conjunto de redes.

Una estrategia que se podría seguir es que el emisor TCP empiece a enviar con una ventana relativamente grande pero no a su máximo valor, esperando que se aproxime finalmente al tamaño que sería proporcionado por la conexión. Esto sin embargo es arriesgado ya que el emisor podría inundar el conjunto de redes con muchos segmentos antes de darse cuenta a partir de los temporizadores que el flujo era excesivo. En lugar de eso, se necesita algún medio para expandir gradualmente la ventana hasta que se reciba una confirmación.

Jacobson [JACO88] recomienda un procedimiento llamado comienzo lento. TCP hace uso de una ventana de congestión, medida en segmentos en lugar de octetos. En cualquier instante de tiempo, la transmisión TCP está condicionada por la siguiente regla:

$$awnd = \text{MIN}[\text{crédito}, cwnd] \quad (17.7)$$

donde

$awnd$ = ventana permitida en segmentos. Éste es el número de segmentos que TCP tiene permitido enviar sin recibir confirmaciones adicionales.

$cwnd$ = ventana de congestión, en segmentos. Ventana utilizada por TCP durante el proceso de aumentar el flujo y para reducir el flujo durante los periodos de congestión.
 $crédito$ = la cantidad de créditos concedidos y no utilizados en la confirmación más reciente, en segmentos. Cuando se recibe una confirmación, este valor se calcula como $ventana/tamaño-de-segmento$, donde $ventana$ es el campo ventana del segmento TCP recibido (la cantidad de datos que la entidad TCP par está dispuesta a aceptar).

Cuando se abre una conexión, la entidad TCP inicializa $cwnd = 1$. Esto es, a TCP sólo se le permite enviar un segmento y después debe esperar una confirmación antes de enviar un segundo segmento. Cada vez que recibe una confirmación, se incrementa el valor de $cwnd$ en una unidad, hasta algún valor máximo.

En realidad, el mecanismo de comienzo lento chequea el conjunto de redes para asegurarse de que no está enviado demasiados segmentos en un ambiente ya congestionado. Conforme van llegando las confirmaciones, a TCP se le permite abrir su ventana hasta que el flujo se controla por las confirmaciones entrantes en lugar de por $cwnd$.

El término *comienzo lento* es un nombre poco apropiado, ya que $cwnd$ crece en realidad exponencialmente. Cuando llega la primera ACK, TCP abre $cwnd$ hasta 2 y puede enviar dos segmentos. Cuando estos dos segmentos se confirman, TCP puede desplazar la ventana 1 segmento por cada ACK que llega e incrementar $cwnd$ en uno por cada ACK. Por lo tanto, en este punto TCP puede enviar cuatro segmentos. Cuando se confirmen estos cuatro segmentos, TCP podrá enviar ocho segmentos.

Ajuste dinámico de la ventana en caso de congestión

Se ha encontrado que el algoritmo de comienzo lento funciona de una forma efectiva para inicializar una conexión. Permite a TCP determinar rápidamente un tamaño de ventana razonable para la conexión. ¿No sería útil la misma técnica cuando hay necesidad de atajar la congestión? En particular, suponga que una entidad TCP inicia una conexión y lo hace mediante el procedimiento de comienzo lento. En determinado punto, antes o después de que $cwnd$ alcance el tamaño de créditos asignados por la otra parte, se pierde un segmento (expira un temporizador). Esto es una señal de que está ocurriendo congestión. Pero no está claro como de sería es la congestión. Por lo tanto, un procedimiento prudente sería inicializar $cwnd$ a 1 y comenzar el procedimiento de comienzo lento de nuevo.

Esto parece un procedimiento razonable y conservativo, pero en realidad no es lo bastante conservativo. Jacobson [JACO88] señala que «es fácil llevar una red a la saturación pero difícil que la red se recupere». En otras palabras, una vez que la congestión ocurre, puede transcurrir un tiempo bastante grande hasta que ésta desaparezca¹. Así, de esta forma el crecimiento exponencial de $cwnd$ bajo el comienzo lento puede ser demasiado agresivo y empeorar la congestión. En lugar de esto, Jacobson propone el uso del comienzo lento para comenzar la conexión, seguido de un crecimiento lineal de $cwnd$. Las reglas son las siguientes. Cuando expira un temporizador:

1. Establecer un umbral de comienzo lento igual a la mitad de la ventana de congestión actual; esto es, hacer $ssthresh = cwnd/2$.
2. Hacer $cwnd = 1$ y ejecutar el procedimiento de comienzo lento hasta que $cwnd = ssthresh$. En esta fase, $cwnd$ se incrementa 1 por cada ACK recibida.
3. Para $cwnd \geq ssthresh$, incrementar $cwnd$ en uno por cada tiempo de ida-y-vuelta.

La Figura 17.14 muestra este comportamiento. Nótese que le lleva 11 tiempos de ida-y-vuelta recuperar el nivel $cwnd$ que inicialmente se consiguió con 4 tiempos de ida-y-vuelta.

¹ Kleinrock se refiere a este fenómeno como un efecto de una cola grande en periodos punta. Véase la Sección 2.7 y la 2.10 de [KLEI76] para una discusión detallada.

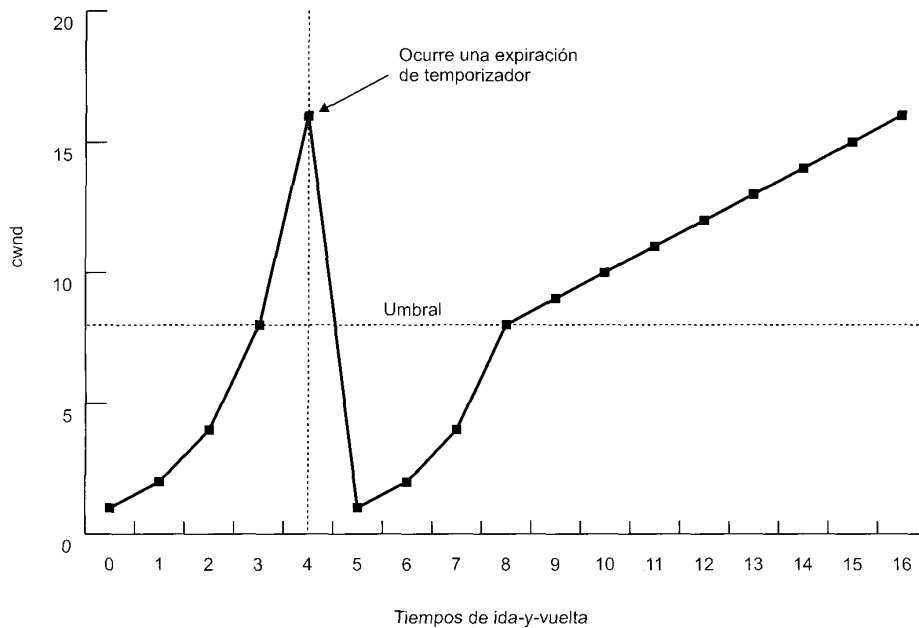


Figura 17.14. Ilustración del comienzo lento y la prevención de la congestión.

17.4. UDP

Además de TCP, existe otro protocolo en la capa de transporte que se usa comúnmente como parte del conjunto de protocolos TCP/IP: el protocolo de datagrama de usuario (UDP, User Datagram Protocol), especificado en el RFC 768. UDP proporciona un servicio no orientado a conexión para los procedimientos de la capa de aplicación. Así, UDP es básicamente un servicio no seguro; la entrega y la protección contra duplicados no están garantizadas. En contrapartida se reduce la información suplementaria del protocolo lo que puede ser adecuado en muchos casos. Como ejemplo de uso de UDP se tiene el contexto de gestión de red, como se describe en el Capítulo 19.

La potencia del enfoque orientado a conexión es clara. Permite características relacionadas con la conexión como son el control de flujo, control de errores y entrega en secuencia. Sin embargo, un servicio no orientado a conexión es más apropiado en algunos contextos. En capas inferiores (interconexión, red) son más robustos (por ejemplo, ver la discusión de la Sección 10.1). Además, representa el «menor denominador común» del servicio que esperan las capas superiores. Pero, incluso a nivel de transporte y superiores existe una justificación para un servicio no orientado a conexión. Existen casos en los que la información suplementaria del establecimiento y mantenimiento de la conexión no están justificados o son contraproducentes. Algunos ejemplos de esto mismo son los siguientes:

- **Recolección de datos de entrada:** supone una actividad periódica o muestreo de datos pasivo, tales como los procedentes de sensores, de informes de auto-test automáticos de equipos de red o de componentes de la red. En una situación de monitorización en tiempo real, la pérdida ocasional de una unidad de datos no causaría ningún desastre ya que la siguiente muestra debería llegar en breves momentos.
- **Diseminar datos de salida:** incluye la difusión de mensajes a los usuarios de la red, el anuncio de un nuevo nodo o el cambio de la dirección de un servicio, y la distribución de los valores de reloj en tiempo real.

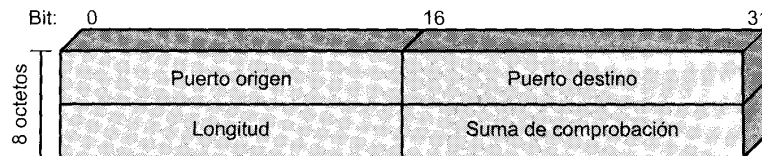


Figura 17.15. Cabecera UDP.

- **Petición-respuesta:** aplicaciones en las que un servidor común proporciona un servicio de transacción a un número de usuarios distribuidos y para lo cual es típico usar una secuencia del tipo petición/respuesta. El uso del servicio se regula en la capa de aplicación y las conexiones a las capas son innecesarias y molestas.
- **Aplicaciones en tiempo real:** tales como voz y teledidáctica, que llevan consigo el requisito de utilizar redundancias y transmisión en tiempo real. Estos requisitos no pueden tener funciones orientadas a conexión tal como es la retransmisión.

Así, tiene sentido que en la capa de transporte tengan cabida tanto servicios orientados a conexión como servicios no orientados a conexión.

UDP se sitúa encima de IP. Ya que es no orientado a conexión, UDP tiene pocas funciones que hacer. Esencialmente, incorpora un direccionamiento a puerto a las capacidades de IP. Esto se ve mejor examinando la cabecera UDP mostrada en la Figura 17.15. La cabecera incluye un puerto origen y un puerto destino. El campo de longitud contiene la longitud del segmento UDP entero, incluyendo la cabecera y los datos. La suma de verificación es el mismo algoritmo usado para TCP e IP. Para UDP, la suma de verificación se aplica al segmento UDP entero más una pseudo-cabecera incorporada a la cabecera UDP cuando se calcula la suma y es la misma que la usada para TCP. Si se detecta un error, el segmento se descarta sin tomar ninguna medida adicional.

El campo de suma de verificación en UDP es opcional. Si no se utiliza, éste se pone todo a cero. Sin embargo, hay que indicar que la suma de verificación de IP se aplica sólo a la cabecera IP y no al campo de datos, que está compuesto, en este caso, de la cabecera UDP y los datos de usuario. Así, si UDP no implementa ningún cálculo de suma de verificación, los datos de usuario no se comprueban.

17.5. LECTURAS RECOMENDADAS

Quizás el mejor tratamiento de las varias estrategias de TCP para el control de flujo y de la congestión se encuentre en [STEV94]. Un artículo fundamental para entender las cuestiones implicadas es el clásico [JACO88].

JACO88 Jacobson, V. «Congestion Avoidance and Control.» *Proceedings, SIGCOMM'88, Computer Communication Review*, August 1988; reprinted in *Computer Communication Review*, January 1995; a slightly revised version is available at ftp.ee.lbl.gov/papers/congavoid.ps.Z.

STEV94 Stevens, W. *TCP/IP Illustrated, Volume 1: The Protocols*. Reading, MA: Addison-Wesley, 1994.

17.6. PROBLEMAS

- 17.1. Es una práctica común en la mayoría de los protocolos de transporte (en realidad, en la mayoría de los protocolos en todas las capas) que los datos y las señales de control se multiplexen sobre

el mismo canal lógico en cada conexión por usuario. Una alternativa es establecer una única conexión de transporte de control entre cada par de entidades de transporte que se comunican. Esta conexión se usaría para transmitir las señales de control de todas las conexiones de los usuarios de transporte entre las dos entidades. Discutir las implicaciones de esta estrategia.

- 17.2. La discusión sobre control de flujo con un servicio de red seguro, referido como mecanismo de presión hacia atrás (*backpressure*), utiliza un protocolo de control de flujo de una capa inferior. Discutir las desventajas de esta estrategia.
- 17.3. Dos entidades de transporte se comunican a través de una red segura. Supongamos que el tiempo normalizado para transmitir un segmento es igual a 1. Suponer que el retardo de propagación extremo-a-extremo es 3 y que la entrega de un segmento recibido al usuario de transporte requiere un tiempo de 2. El emisor tiene inicialmente concedido un crédito de siete segmentos. El receptor utiliza un criterio de control de flujo conservativo y actualiza su asignación de créditos a cada oportunidad. ¿Cuál es el máximo rendimiento alcanzable?
- 17.4. Dibuje un diagrama similar al de la Figura 17.5 para las siguientes condiciones (suponer un servicio de red seguro y con secuenciamiento):
 - a) Cierre de la conexión: activo/pasivo.
 - b) Cierre de la conexión: activo/activo.
 - c) Rechazo de conexión.
 - d) Proceso de cancelar una conexión: un usuario emite un OPEN a un usuario que está escuchando y entonces emite un CLOSE antes de que se intercambie algún dato.
- 17.5. Con un servicio de red seguro y con secuenciamiento, ¿son estrictamente necesarios los números de secuencia de los segmentos? ¿Qué capacidad, si hay alguna, se pierde sin ellos?
- 17.6. Considere un servicio de red orientado a conexión que sufre un reinicio. ¿Cómo podría ser tratado por un protocolo de transporte que supone que el servicio de red es seguro excepto en el caso de un reinicio?
- 17.7. La discusión del criterio de retransmisión hizo referencia a tres problemas asociados con el cálculo dinámico del valor del temporizador. ¿Qué modificaciones del criterio ayudarían para aliviar estos problemas?
- 17.8. Considere un protocolo de transporte que usa un servicio de red orientado a conexión. Suponga que ese protocolo de transporte utiliza un esquema de asignación de créditos para el control de flujo y que el protocolo de red usa un esquema de ventana deslizante. ¿Qué relación, si existe, debería haber entre la ventana dinámica del protocolo de transporte y la ventana fija del protocolo de red?
- 17.9. En una red que tiene un tamaño máximo de paquete de red de 128 bytes, un tiempo de vida máximo de 30 s y un número de secuencia de paquetes de 8 bits, ¿cuál es la máxima tasa de transmisión de datos por conexión?
- 17.10. ¿Es posible un interbloqueo utilizando un diálogo en dos sentidos en lugar de un diálogo en tres sentidos? Dé un ejemplo o demuéstrela.
- 17.11. Debajo se enumeran cuatro estrategias que se pueden utilizar para proporcionar a un usuario de transporte las direcciones de un usuario de transporte destino. Para cada una, describa una analogía con un usuario del servicio de correos.
 - a) Conocer la dirección de antemano.
 - b) Hacer uso de una dirección «bien-conocida».

- c) Utilizar un servidor de nombres.
 - d) Las direcciones se generan cuando se realiza la petición.
- 17.12.** En un esquema de créditos para control de flujo, ¿qué provisión de créditos se puede hacer para la asignación de créditos que se pierdan o se desordenen durante la transmisión?
- 17.13.** ¿Qué ocurre en la Figura 17.4 si llega un SYN mientras el usuario solicitado está en el estado CERRADO? ¿Hay alguna forma de llamar la atención del usuario cuando éste no está escuchando?
- 17.14.** Normalmente, el campo ventana en la cabecera TCP da una asignación de créditos en octetos. Cuando se utiliza la opción de Escalado de Ventana, el valor del campo Ventana se multiplica por 2^F , donde F es el calor de la opción de escalado de ventana. El valor máximo de F que acepta TCP es 14. ¿Por qué se limita esta opción a 14?
- 17.15.** Una dificultad con la estimación original de SRTT de TCP es la elección de un valor inicial. En ausencia de un conocimiento especial de las condiciones de la red la opción típica es elegir un valor arbitrario, tal como 3 segundos, y esperar que esto converja rápidamente a un valor preciso. Si la estimación es demasiado pequeña, TCP llevará a cabo retransmisiones innecesarias. Si es demasiado grande, TCP esperará un periodo grande de tiempo antes de retransmitir si el primer segmento se perdió. También, la convergencia puede ser lenta, como indica este problema.
- a) Elegir $\alpha = 0,85$ y $SRTT(0) = 3$ segundos y suponer que todos los valores medidos son iguales a 1 segundo y que no se pierden paquetes. ¿Cuál es el valor de $SRTT(9)$? *Sugerencia:* la Ecuación 17.3 se puede reescribir para simplificar los cálculos, utilizando la expresión $(1 - \alpha^n)/(1 - \alpha)$.
 - b) Sea ahora $SRTT(0) = 1$ segundo y suponga que los valores medidos de RTT son 3 segundos y que no se pierden paquetes. ¿Cuál es el valor de $SRTT(19)$?
- 17.16.** Una implementación pobre del esquema de ventana deslizante de TCP puede llevar a un rendimiento extremadamente pobre. Existe un fenómeno conocido como «síndrome de la ventana absurda» (SWS, Silly Window Syndrome), que puede fácilmente causar una degradación en el rendimiento en varias decenas. Como ejemplo de SWS, considere una aplicación que está ocupada en la transferencia de un fichero largo y que TCP está transfiriendo el fichero en segmentos de 200 octetos. El receptor inicialmente asigna un crédito de 1.000. El emisor agota esta ventana con 5 segmentos de 200 octetos. Ahora suponga que el receptor devuelve una confirmación por cada segmento y proporciona un crédito adicional de 200 octetos por cada segmento recibido. Desde el punto de vista del receptor, esto de nuevo expande la ventana hasta 1.000 octetos. Sin embargo, desde el punto de vista del emisor, si la primera confirmación llega después de que se han enviado 5 segmentos solamente hay disponible una venta de 200 octetos solamente. Suponga que en el mismo punto, el receptor calcula una ventana de 200 octetos pero sólo tiene 50 octetos para enviar cuando alcanza un punto de «carga». Por lo tanto, él envía 50 octetos en un segmento, seguido de 150 octetos en el siguiente segmento y reanuda la transmisión de segmentos de 200 octetos. ¿Qué podría ahora ocurrir que de lugar a un problema en el rendimiento? Plantee el SWS en términos más generales.
- 17.17.** TCP impone que tanto el receptor como el emisor incorporen mecanismos para tratar el SWS.
- a) Sugiera una estrategia para el receptor. *Sugerencia:* suponga que el receptor «miente» sobre la capacidad de memoria temporal de que se dispone bajo ciertas circunstancias. Plantee una regla razonable grosso modo para esto.
 - b) Sugiera una estrategia para el emisor. *Sugerencia:* considere la relación entre la ventana máxima posible de envío y lo que hay disponible para enviar.

- 17.18.** En la Ecuación (17.5), reescriba la definición de $SRTT(K + 1)$ para que sea función de $SERR(K + 1)$. Interprete el resultado.
- 17.19.** Una entidad TCP abre una conexión y utiliza un comienzo lento. Aproximadamente, ¿cuántos tiempos de ida y vuelta se necesitan antes que TCP pueda enviar N segmentos?
- 17.20.** Aunque el comienzo lento con la prevención de congestión es una técnica efectiva para tratar la congestión, puede resultar en tiempos de recuperación grandes en redes de capacidad alta, como demuestra este problema:
- a)** Suponga un retardo de ida y vuelta de 60 ms (lo que podría ocurrir a través de un continente) y un enlace con un ancho de banda disponible de 1 Gbps y un tamaño de segmento de 576 octetos. Determine el tamaño de ventana necesario para mantener lleno el cauce y el tiempo que tardaría en alcanzar el tamaño de ventana después de una expiración utilizando el criterio de Jacobson.
 - b)** Repita (a) para un tamaño de venta de 16 Kbytes.
- 17.21.** ¿Por qué es necesario UDP? ¿Por qué no puede un programa de usuario acceder directamente a IP?

Seguridad en redes

18.1. Requisitos y amenazas a la seguridad

- Ataques pasivos
- Ataques activos

18.2. Privacidad con cifrado convencional

- Cifrado convencional
- Algoritmos de cifrado
- Localización de los dispositivos de cifrado
- Distribución de claves
- Relleno de tráfico

18.3. Autenticación de mensajes y funciones de dispersión («hash»)

- Técnicas de autenticación de mensajes
- Funciones de dispersión seguras
- La función de dispersión segura SHA-1

18.4. Cifrado de clave pública y firmas digitales

- Cifrado de clave pública
- Firmas digitales
- El algoritmo de cifrado de clave pública RSA
- Gestión de claves

18.5. Seguridad con IPv4 e IPv6

- Aplicaciones de IPSec
- El ámbito de IPSec
- Asociaciones de seguridad
- Modos de transporte y modos túnel
- Cabecera de autenticación
- Encapsulado de seguridad de la carga útil
- Gestión de claves

18.6. Lecturas recomendadas y páginas Web

18.7. Problemas



- Las amenazas a la seguridad de red se dividen en dos categorías: **amenazas pasivas**, llamadas a veces escuchas, y que suponen el intento de un atacante de obtener información relativa a una comunicación; y **amenazas activas** que suponen alguna modificación de los datos transmitidos o la creación de transmisiones falsas.
- Hasta ahora la herramienta automática más importante para la seguridad en red y de la comunicación es el **cifrado**. Con el cifrado convencional, dos partes comparten una clave de cifrado/descifrado. El principal reto del cifrado convencional es la distribución y la protección de las claves. Un esquema de cifrado de clave pública implica dos claves, una para el cifrado y la otra para el descifrado. Una de las claves es privada de la parte que genera el par de claves y la otra se hace pública.
- El cifrado convencional y el cifrado de clave pública se suelen combinar en aplicaciones de red seguras. El cifrado convencional se utiliza para cifrar los datos transmitidos, con una clave utilizada una sola vez o clave de sesión temporal. La clave de sesión la puede distribuir un centro de distribución de claves de confianza o ser transmitida cifrada utilizando un cifrado de clave pública. El cifrado de clave pública también se utiliza para crear firmas digitales, que pueden autenticar la fuente de los mensajes transmitidos.
- Una mejora en la seguridad utilizada con IPv4 e IPv6, llamada IPSec, proporciona mecanismos de confidencialidad y autenticación.



Los requisitos en la **seguridad de la información** dentro de un organismo han sufrido principalmente dos cambios en las últimas décadas. Antes de que se extendiera la utilización de los equipos de procesamiento de datos, la seguridad de la información, que era de valor para una institución se conseguía fundamentalmente por medios físicos y administrativos. Como ejemplo del primer medio es el uso de cajas fuertes con combinación de apertura para almacenar documentos confidenciales. Un ejemplo del segundo es el uso de procedimientos de investigación de personal durante la fase de contratación.

Con la introducción de los computadores, fue evidente la necesidad de herramientas automáticas para proteger ficheros y otra información almacenada en los computadores. Éste es especialmente el caso de los sistemas multiusuario, como con los sistemas de tiempo compartido, y la necesidad es más aguda para sistemas a los que se puede acceder desde teléfonos públicos o redes de datos. El término genérico del campo que trata las herramientas diseñadas para proteger los datos y frustrar a los piratas informáticos es **seguridad en computadores**. Aunque éste es un tópico muy importante, está fuera del ámbito de este libro y será tratado muy brevemente.

El segundo cambio relevante, que ha afectado a la seguridad, es la introducción de sistemas distribuidos y la utilización de redes y facilidades de comunicación para transportar datos entre terminales de usuario y computadores, y de computador a computador. Las medidas de **seguridad en red** son necesarias para proteger los datos durante su transmisión y garantizar que los datos transmitidos sean auténticos.

Virtualmente la tecnología esencial subyacente en todas las redes automáticas y las aplicaciones de seguridad en computadores es el cifrado. Existen dos técnicas fundamentales en uso: cifrado convencional, también conocido como cifrado simétrico, y el cifrado con clave pública, también conocido como cifrado asimétrico. Conforme examinemos las diversas técnicas de seguridad en red, se explorarán los dos tipos de cifrado.

Este capítulo comienza con una visión global de los requisitos de seguridad en red. A continuación, se examinará el cifrado convencional y su utilización para proporcionar privacidad. A esto le sigue una

discusión sobre autenticación de mensajes. Se examina el uso del cifrado de clave pública y las firmas digitales. El capítulo termina con un examen de las características de seguridad en IPv4 e IPv6.

18.1. REQUISITOS Y AMENAZAS A LA SEGURIDAD

Para ser capaz de entender los tipos de amenazas a la seguridad que existen, conviene definir los requisitos en seguridad. La seguridad en computadores y en redes implica tres requisitos:

- **Secreto:** requiere que la información en un computador sea accesible para lectura sólo por los entes autorizados. Este tipo de acceso incluye la impresión, mostrar en pantalla y otras formas de revelación que incluye cualquier forma de dar a conocer la existencia de un objeto.
- **Integridad:** requiere que los recursos de un computador sean modificados solamente por entes autorizados. La modificación incluye escribir, cambiar, cambiar de estado, suprimir y crear.
- **Disponibilidad:** requiere que los recursos de un computador estén disponibles a los entes autorizados.

Una clasificación útil de las agresiones a la seguridad en red se hace en términos de agresiones pasivas y agresiones activas (Figura 18.1).

ATAQUES PASIVOS

Las agresiones pasivas son del tipo de las escuchas, o monitorizaciones, de las transmisiones. La meta del oponente es obtener información que está siendo transmitida. Existen dos tipos de agresiones: divulgación del contenido de un mensaje y análisis del tráfico.

La **divulgación del contenido de un mensaje** se entiende fácilmente. Una conversación telefónica, un mensaje de correo electrónico, un fichero transferido puede contener información sensible o confidencial. Así, sería deseable prevenir que el oponente se entere del contenido de estas transmisiones.

El segundo tipo de agresión pasiva, el **análisis del tráfico**, es más sutil. Suponga que tenemos un medio de enmascarar el contenido de los mensajes u otro tipo de tráfico de información, aunque se capturan los mensajes, no se podría extraer la información del mensaje. La técnica más común para enmascarar el contenido es el cifrado. Pero incluso si tenemos protección de cifrado, el oponente podría ser capaz de observar los modelos de estos mensajes. El oponente podría determinar la localización y la identidad de los computadores que se están comunicando y observar la frecuencia y la longitud de los mensajes intercambiados. Esta información puede ser útil para extraer la naturaleza de la comunicación que se está realizando.

Los ataques pasivos son muy difíciles de detectar ya que no implican la alteración de los datos. Sin embargo, es factible prevenir el éxito de estas agresiones. Así, el énfasis para tratar estas agresiones está en la prevención antes que la detección.

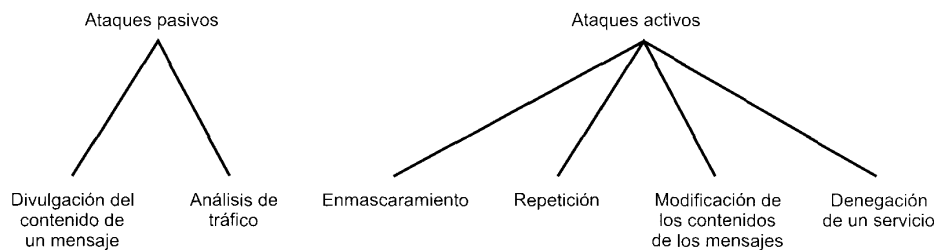


Figura 18.1. Agresiones a la seguridad de red activas y pasivas.

ATAQUES ACTIVOS

Los ataques activos suponen alguna modificación del flujo de datos o la creación de flujos falsos y subdividen en 4 categorías: enmascaramiento, repetición, modificación de mensajes y denegación de servicio.

Un **enmascaramiento** tiene lugar cuando una entidad pretende ser otra entidad diferente. Una acción de enmascaramiento normalmente incluye una de las otras formas de agresión activa. Por ejemplo, se puede captar una secuencia de autenticación y reemplazarla por otra secuencia de autenticación válida, así se habilita a otra entidad autorizada con pocos privilegios a obtener privilegios extras suplantando a la entidad que los tiene.

La **repetición** supone la captura pasiva de unidades de datos y su retransmisión subsecuente para producir un efecto no autorizado.

La **modificación de mensajes** significa sencillamente que alguna porción de un mensaje legítimo se altera, o que el mensaje se retrasa o se reordena para producir un efecto no autorizado. Por ejemplo, un mensaje con un significado «Permitir a John Smith leer el fichero confidencial de cuentas» se modifica para tener el significado «Permitir a Fred Brown leer el fichero confidencial de cuentas».

La **denegación de un servicio** previene o inhibe el uso o gestión normal de las facilidades de comunicación. Esta agresión puede tener un objetivo específico: por ejemplo, una entidad puede suprimir todos los mensajes dirigidos a un destino particular (por ejemplo, al servicio de vigilancia de seguridad). Otro tipo de denegación de servicio es la perturbación sobre una red completa, deshabilitándola o sobrecargándola con mensajes de forma que se degrade su rendimiento.

Las agresiones activas presentan características opuestas a las agresiones pasivas. Mientras una agresión pasiva es difícil de detectar, existen medidas disponibles para prevenirlas. Por otro lado, es bastante difícil prevenir una agresión activa, ya que para hacerlo se requeriría protección física constante de todos los recursos y de todas las rutas de comunicación. Por consiguiente, la meta es detectarlas y recuperarse de cualquier perturbación o retardo causados por ellas. Ya que la detección tiene un efecto disuasivo, también puede contribuir a la prevención.

18.2. PRIVACIDAD CON CIFRADO CONVENCIONAL

La técnica universal para proporcionar privacidad en los datos transmitidos es el cifrado convencional. Esta sección examina primero los conceptos básicos del cifrado convencional, y sigue con una discusión de las dos técnicas de cifrado convencional más populares: DES y DES triple. Después se examinarán las aplicaciones de estas técnicas para alcanzar la privacidad.

CIFRADO CONVENCIONAL

El cifrado convencional, también llamado cifrado simétrico o de clave única, era el único tipo de cifrado en uso antes de la introducción del cifrado de clave pública a finales de la década de los 70. El cifrado convencional ha sido utilizado para las comunicaciones secretas por incontables individuos y grupos, desde Julio César hasta la fuerza alemana de los U-boat y actualmente los diplomáticos, militares y los comerciantes. Es todavía el cifrado más utilizado mundialmente de los dos tipos de cifrado.

Un esquema de cifrado convencional tiene cinco ingredientes (Figura 18.2):

- **Texto nativo («plaintext»):** es el mensaje original o datos que actúan como entrada al algoritmo.
- **Algoritmo de cifrado:** el algoritmo de cifrado lleva a cabo varias substituciones y transformaciones en el texto nativo.
- **Clave secreta:** la clave secreta es también una entrada al algoritmo de cifrado. Las substituciones y transformaciones exactas realizadas por el algoritmo dependen de la clave.

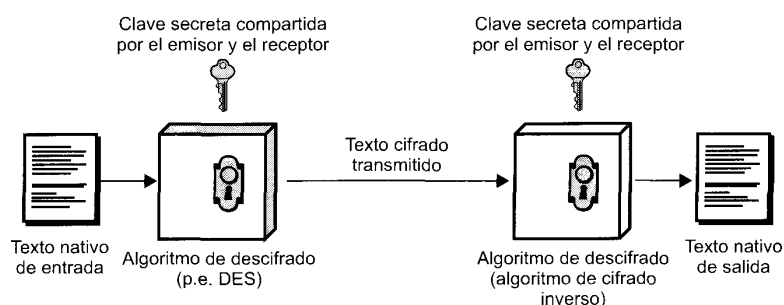


Figura 18.2. Modelo simplificado del cifrado convencional.

- **Texto cifrado:** es el mensaje aleatorio que se produce en la salida. Depende del texto nativo y de la clave secreta. Para un mensaje dado, dos claves diferentes producen dos textos cifrados diferentes.
- **Algoritmo de descifrado:** es esencialmente el algoritmo de cifrado ejecutado al revés. Toma como entradas el texto cifrado y la clave secreta y produce el texto nativo original.

Existen dos requisitos para la utilización segura del cifrado convencional:

1. Se necesita un algoritmo de cifrado robusto. Como mínimo, es de desear un algoritmo tal que si el oponente conoce el algoritmo y tiene acceso a más de un texto cifrado, sea incapaz de descifrar el texto o adivinar la clave. Este requisito se puede enunciar de una forma más estricta: el oponente debería ser incapaz de descifrar el texto o descubrir la clave incluso si él o ella posee un determinado número de texto cifrados junto a los textos nativos que producen cada texto cifrado.
2. El emisor y el receptor deben haber obtenido las copias de la clave secreta de una forma segura y deben mantenerla en secreto. Si alguien puede descubrir la clave y conoce el algoritmo, todas las comunicaciones que utilicen esta clave pueden ser leídas.

Existen dos enfoques generales para atacar el esquema de cifrado convencional. El primer ataque se conoce como **criptoanálisis**. El ataque de criptoanálisis se basa en la naturaleza del algoritmo más quizás algún conocimiento de las características generales del texto nativo o incluso de algunos pares texto nativo-texto cifrado. Este tipo de ataque explota las características del algoritmo para intentar deducir un texto nativo específico o deducir la clave que se está utilizando. Si el ataque tiene éxito en deducir la clave, el efecto es catastrófico: todos los mensajes cifrados antiguos y futuros con esa clave están comprometidos.

El segundo método, conocido como ataque de **fuerza bruta**, es intentar cada clave posible en un trozo de texto cifrado hasta que se obtenga una traducción inteligible del texto nativo. La Tabla 18.1 muestra cuánto tiempo se necesita para el caso de varios tamaños de clave. La tabla muestra los resultados para cada tamaño de clave, suponiendo que se tarda 1 μ s. en llevar a cabo un único descifrado, que es un orden de magnitud razonable para los computadores de hoy en día. Con el uso de una arquitectura paralela masiva de microcomputadores, sería posible alcanzar velocidades de procesamiento

Tabla 18.1. Tiempo promedio necesario para una búsqueda de clave exhaustiva

Tamaño de la clave (bits)	Número de claves alternativas	Tiempo necesario a 1 cifrado/ μ s	Tiempo necesario a 10^6 cifrados/ μ s
32	$2^{32} = 4,3 \times 10^9$	$2^{31} \mu\text{s} = 35,8$ minutos	2,15 milisegundos
56	$2^{56} = 7,2 \times 10^{16}$	$2^{55} \mu\text{s} = 1.142$ años	10,01 horas
128	$2^{128} = 3,4 \times 10^{38}$	$2^{127} \mu\text{s} = 5,4 \times 10^{24}$ años	$5,4 \times 10^{18}$ años
168	$2^{168} = 3,7 \times 10^{50}$	$2^{167} \mu\text{s} = 5,9 \times 10^{36}$ años	$5,9 \times 10^{30}$ años

de varias órdenes de magnitud superiores. La última columna de la tabla considera los resultados de un sistema que pudiera procesar 1 millón de claves por microsegundo. Como se puede ver, a este nivel de rendimiento, ya no se puede considerar computacionalmente segura una clave de 56 bits.

ALGORITMOS DE CIFRADO

Los algoritmos de cifrado usados más comúnmente son los cifradores de bloque. Un cifrador de bloque procesa una entrada de texto nativo en bloques de tamaño fijo, y produce un bloque de texto cifrado de igual tamaño para cada bloque de texto nativo. Los dos algoritmos convencionales más importantes, cifradores de bloque, son el DES y el TDEA.

El estándar de cifrado de datos (DES, Data Encryption Standard)

El esquema de cifrado más utilizado mundialmente se define en el estándar de cifrado de datos (DES) adoptado en 1977 por el Buró Nacional de Estándares, ahora el Instituto Nacional de Estándares y Tec-

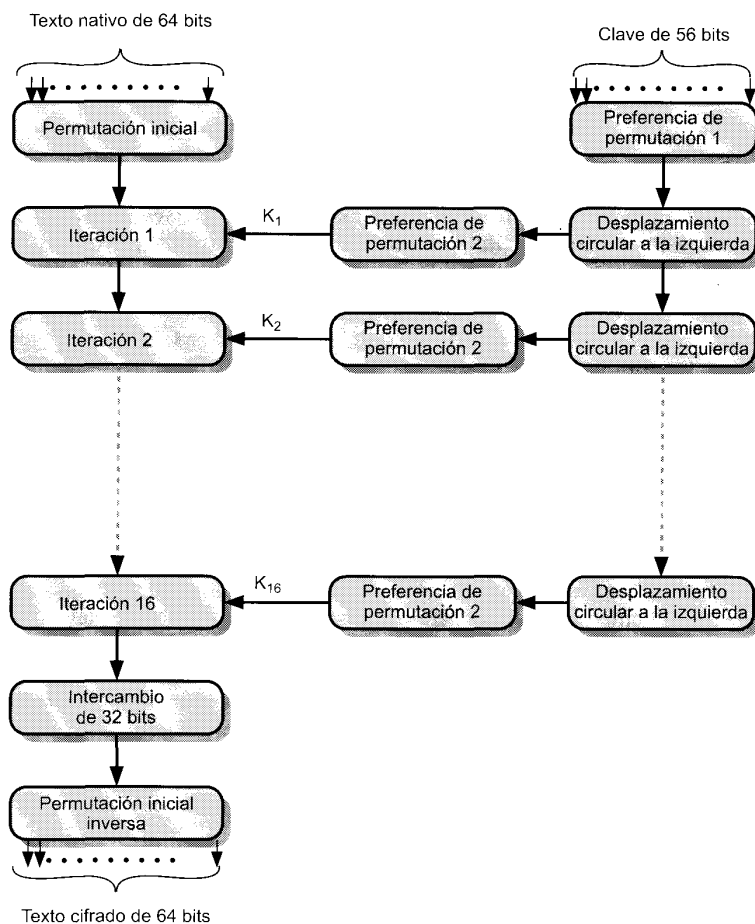


Figura 18.3. Esquema general del algoritmo de cifrado DES.

nología (NIST, National Institute of Standards and Technology»), como Estándar Federal de Procesamiento de la Información 46 (FIPS PUB 46). En 1994, el NIST «reafirmó» a DES para su uso federal por otros 5 años en FIPS PUB-46-2. El algoritmo se denomina algoritmo de Cifrado de Datos (DEA, Data Encryption Algorithm).

El esquema global del cifrado DES se muestra en la Figura 18.3. El texto nativo debe tener una longitud de 64 bits y la clave 56 bits; los textos nativos más grandes se procesan en bloques de 64 bits.

El lado izquierdo de la figura muestra que el procesamiento del texto nativo se realiza en tres fases. Primero, los 64 bits del texto nativo se transforman por medio de una permutación inicial (IP) que reordena los bits para producir la entrada permutada. A esto le sigue una fase que consta de 16 iteraciones de la misma función. La salida de la última iteración (la 16) consta de 64 bits que son función del texto nativo y la clave. La mitad izquierda y la derecha se intercambian para producir la salida previa. Finalmente, la salida previa se permuta con IP^{-1} , que es la inversa de la función de permutación inicial, para producir los 64 bits del texto cifrado.

La parte derecha de la Figura 18.3 muestra la forma cómo se usan los 56 bits de la clave. Inicialmente, la clave se transforma por una función de permutación. Después, para cada una de las 16 iteraciones, se produce una subclave (K_i) por medio de un desplazamiento circular y una permutación. La función de permutación es la misma para cada iteración, pero se produce una subclave diferente debido al desplazamiento repetido de los bits de la clave.

La Figura 18.4 muestra con más detalle el algoritmo para una iteración. Los 64 bits permutados de entrada pasan a través de las 16 iteraciones, produciendo un valor de 64 bits intermedios a la conclusión de cada iteración. La mitad izquierda y derecha de cada valor intermedio de 64 bits se tratan como

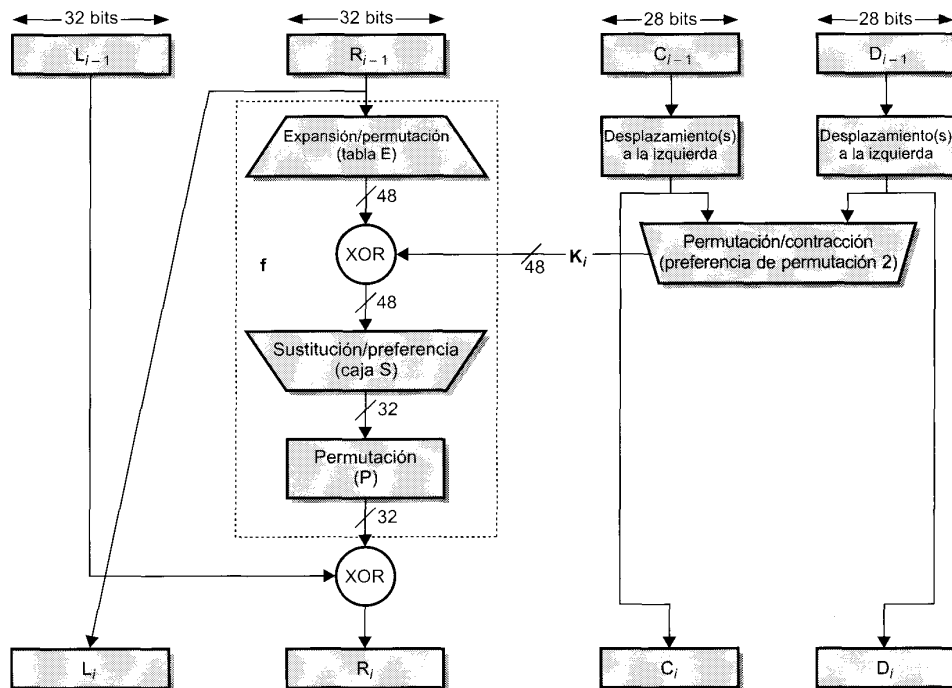


Figura 18.4. Iteración simple del algoritmo DES.

cantidades de 32 bits separadas, rotuladas L (izquierda) y R (derecha). El procesamiento global de cada iteración se puede resumir en las siguientes fórmulas:

$$\begin{aligned}L_i &= R_{i-1} \\ R_i &= L_{i-1} \oplus (R_{i-1}, K_i)\end{aligned}$$

donde $\oplus$ denota la función XOR bit-a-bit.

Así, la parte izquierda de la salida de una iteración (L_i) es simplemente igual a la parte derecha de la entrada a esa iteración (R_{i-1}). La parte derecha de la salida (R_i) es la función OR-exclusiva de (L_{i-1}) y una función compleja de R_{i-1} y K_i . Esta función compleja supone operaciones de permutación y sustitución. La operación de sustitución, representada por tablas llamadas «cajas S», simplemente traduce cada combinación de entrada de 48 bits en un modelo particular de 32 bits.

Volviendo a la Figura 18.3, vemos que la clave de 56 bits usada como entrada al algoritmo, sufre primero una permutación. La clave resultante de 56 bits se trata entonces como dos cantidades de 28 bits, rotuladas como C_0 y D_0 . En cada iteración, C y D sufren de forma separada un desplazamiento circular a la izquierda, o rotación, de 1 o 2 bits. Estos valores desplazados sirven como entrada para la siguiente iteración. También se utilizan como entrada a otra función de permutación, que produce una salida de 48 bits que sirve como entrada a la función $f(R_{i-1}, K_i)$.

El proceso de descifrado con DES es esencialmente el mismo que el proceso de cifrado. La regla es como sigue: usar un texto cifrado como entrada al algoritmo DES, pero usar la clave K_i en orden inverso. Esto es, utilizar K_{16} en la primera iteración, K_{15} en la segunda iteración y así hasta que se utilice K_1 en la iteración 16 y última.

La potencia de DES

Ya desde finales de la década de los 70, los expertos advirtieron que los días de DES como algoritmo seguro estaban contados y era sólo cuestión de tiempo el aumento de la velocidad de los procesadores y la caída del coste del hardware que hicieran una cuestión simple romper DES fácilmente. El paciente se declaró muerto en julio de 1998, cuando la Fundación de la Fronteras Electrónicas (EFF, Electronic Frontier Foundation) anunció que había roto el nuevo reto DES utilizando una máquina sabotadora de DES de propósito específico que se construyó por menos de 250.000 dólares. El ataque duró menos de tres días. La EFF ha publicado una descripción detallada de la máquina, permitiendo que otros construyan su propio sabotador [EFF98]. Y, por supuesto, los precios del hardware continuarán cayendo y la velocidad aumentando, haciendo de DES un algoritmo sin ningún valor.

Afortunadamente, existen varias alternativas disponibles en el mercado. A continuación se examina la alternativa más utilizada.

DEA Triple

El DES Triple fue propuesto por primera vez por Tuchman [TUCH79] y fue la primera normalización para aplicaciones comerciales en el estándar ANSI X9.17 de 1985. TDEA se incorporó como una parte del Estándar de Cifrado de Datos en 1999, con la publicación de FIPS PUB46-3.

El TDEA utiliza dos claves y tres ejecuciones del algoritmo DES. La función sigue una secuencia cifrado-descifrado-cifrado (EDE):

$$C = E_{K_3}[D_{K_2}[E_{K_1}[P]]]$$

donde

C = texto cifrado

P = texto nativo

$E_K[X]$ = cifrado de X utilizando K

$D_K[Y]$ = descifrado de Y utilizando K

La utilización del descifrado en la segunda etapa no tiene significado criptográfico. Su única ventaja es que permite a los usuarios de DES Triple descifrar datos de usuarios del antiguo DES:

$$C = E_{K_1}[D_{K_2}[E_{K_1}[P]]] = E_{K_1}[P]$$

Con tres claves distintas, TDEA tiene una longitud de clave efectiva de 168 bits. FIPS 46-3 también permite el uso de dos claves haciendo $K_1 = K_3$; esto proporciona una longitud de clave de 112 bits. FIPS 46-3 incluye las siguientes indicaciones para TDEA:

- TDEA es el algoritmo elegido para el cifrado convencional aprobado por FIPS.
- El DEA original, que utiliza una clave única de 56 bits, se permite sólo por el estándar para sistemas que previamente disponían de él. Los nuevos procedimientos deberían permitir TDEA.
- Los organismos gubernamentales con sistemas DEA antiguos se les anima a hacer una transición a TDEA.
- Se anticipa que TDEA y el Estándar de Cifrado Avanzado (AES, Advanced Encryption Standard) coexistirán como algoritmos aprobados por FIPS, permitiendo una transición gradual a AES.

AES es el nuevo estándar de cifrado convencional que está desarrollándose por el Instituto Nacional de Estándares y Tecnología. Se pretende que AES proporcione una seguridad en el cifrado robusta en el futuro próximo. Su característica principal es que utiliza un tamaño de bloque de 128 bits (comparado con los 64 bits de DEA y TDEA) y una longitud de clave de al menos 128 bits. Sin embargo, faltan unos cuantos años antes de que se finalice y antes de que se vea sujeto al criptoanálisis intenso al que se vio sometido DEA. La ventaja de AES es que será probablemente más fácil de implementar y rápido en ejecutarse, en hardware y software, que TDEA.

LOCALIZACIÓN DE LOS DISPOSITIVOS DE CIFRADO

El enfoque más potente y más común en contra de los ataques a la seguridad de red es el cifrado. Si se va a utilizar el cifrado en contra de estas amenazas, entonces necesitamos decidir qué vamos a cifrar y dónde se va a situar la máquina de cifrado. Como muestra la Figura 18.5, hay dos alternativas fundamentales: cifrado de enlace y cifrado extremo-a-extremo.

Con el cifrado de enlace, cada enlace de comunicación vulnerable se equipa en ambos extremos con un dispositivo de cifrado. Así, todo el tráfico a través de los enlaces de comunicaciones se protege. Aunque esto requiere muchos dispositivos de cifrado en redes grandes, el valor de esta opción está claro. Una desventaja es que el mensaje debe ser descifrado cada vez que entra un paquete en un conmutador; esto es así ya que el conmutador debe leer la dirección (número de circuito virtual) en la cabecera del paquete para encaminarlo. De esta forma, el mensaje es vulnerable en cada nodo. Si la red es de conmutación de paquetes pública, el usuario no tiene control sobre la seguridad en los nodos.

Con un cifrado extremo-a-extremo, el proceso de cifrado se realiza en los dos sistemas finales. El computador o terminal origen cifra los datos. Los datos cifrados se transmiten sin alterarlos a través de la red hasta el computador o terminal destino. El destino comparte una clave con el origen y es, por lo tanto, capaz de descifrar los datos. Esta técnica parece proteger la transmisión contra agresiones en los enlaces o conmutadores de red. Hay, sin embargo, un punto débil.

Considere la siguiente situación. Un computador se conecta a una red de conmutación de paquetes X.25, establece un circuito virtual a otro computador y se prepara para enviar datos al otro computador utilizando un cifrado extremo-a-extremo. Los datos se transmiten por esa red en la forma de paquetes, que constan de una cabecera y algunos datos de usuario. ¿Qué parte de cada paquete cifrará el computador? Supongamos que el computador cifra el paquete entero, incluyendo la cabecera. Esto no funcionará

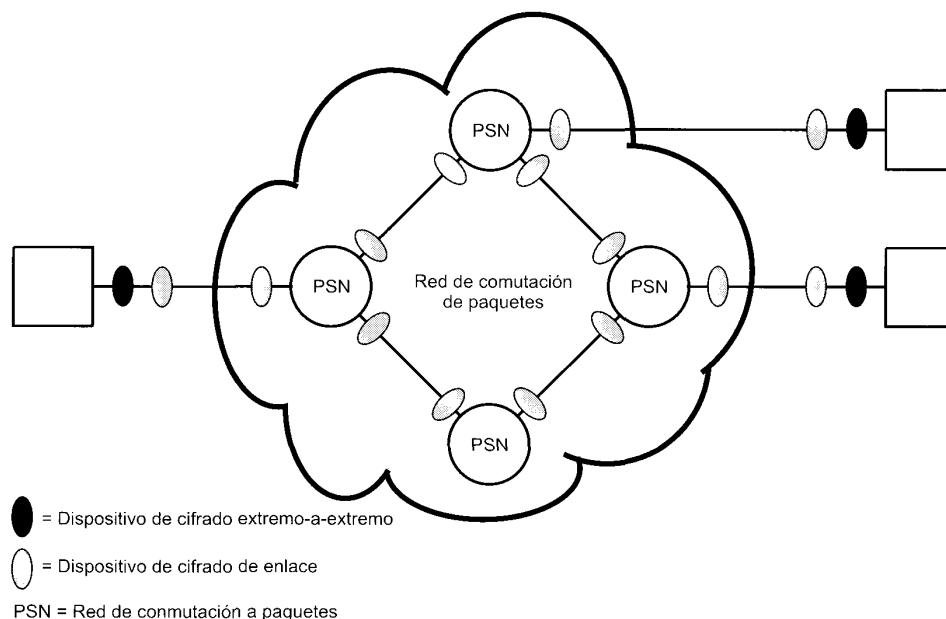


Figura 18.5. Cifrado a través de una red de conmutación a paquetes.

ya que, recuerde, sólo el otro computador puede descifrar el paquete. El nodo de conmutación recibirá el paquete cifrado y no será capaz de leer la cabecera. Por lo tanto, no será capaz de encaminar el paquete. De esto se concluye que el computador sólo puede cifrar la parte de datos de usuario y no la parte de cabecera, para que ésta pueda ser leída por la red.

Así, con el cifrado extremo-a-extremo, los datos de usuario están seguros. Sin embargo, el modelo de tráfico no lo está, ya que las cabeceras de los paquetes se transmiten sin cifrarlas. Para alcanzar un mayor grado de seguridad, se necesita cifrado de enlace y extremo-a-extremo, como se muestra en la Figura 18.5.

Para resumir, cuando se utilizan ambas formas, el computador cifra la parte de datos de usuario usando una clave de cifrado extremo-a-extremo. Después se cifra el paquete entero usando una clave de cifrado de enlace. Conforme el paquete viaja por la red, cada nodo conmutador descifra el paquete utilizando una clave de cifrado de enlace para poder leer la cabecera y luego cifra de nuevo el paquete entero para enviarlo al siguiente enlace. Ahora el paquete está seguro excepto durante el tiempo en el que el paquete está en la memoria del nodo de conmutación, en el que la cabecera está desprotegida.

DISTRIBUCIÓN DE CLAVES

Para que funcione el cifrado convencional, las dos partes que intercambian datos deben tener la misma clave y ésta debe ser protegida para que no la conozcan otros. Además, es deseable realizar normalmente cambios de la clave para limitar la cantidad de datos comprometidos si una agresión aprende la clave. Por lo tanto, la potencia de cualquier sistema de cifrado se apoya en una técnica de distribución de claves, un término que se refiere a los medios para distribuir una clave a dos partes que quieren intercambiar datos, impidiendo que otros vean la clave. La distribución de claves se puede conseguir de varias formas. Para dos partes A y B:

1. A puede seleccionar una clave y entregarla físicamente a B.

2. Una tercera parte selecciona la clave y la entrega físicamente a A y B.
3. Si A y B han utilizado previamente y recientemente una clave, una de las partes podría transmitir la nueva clave a la otra cifrada utilizando la clave previa.
4. Si A y B tienen cada uno una conexión cifrada a una tercera parte C, C podría entregar una clave a través de los enlaces cifrados a A y a B.

Las opciones 1 y 2 exigen una entrega manual de una clave. Éste es un requisito razonable para el cifrado de enlace ya que cada dispositivo de cifrado de enlace sólo va a intercambiar datos con su pareja en el otro extremo de enlace. Sin embargo, para cifrado extremo-a-extremo, la entrega manual es difícil. En un sistema distribuido, cualquier terminal o computador se ve envuelto en intercambios con muchos otros terminales o computadores durante mucho tiempo. Así, cada dispositivo necesita varias claves, suministradas dinámicamente. El problema es especialmente difícil en sistemas distribuidos a través de una gran área.

La opción 3 es una posibilidad válida tanto para el cifrado de enlace como el de extremo-a-extremo, pero si un agresor tiene éxito al conseguir una clave, todas las claves siguientes serán reveladas. Incluso si se hacen cambios frecuentes en la clave de cifrado de enlace, éstos se deberían hacer manualmente. La opción 4 es la preferible para proporcionar claves de cifrado extremo-a-extremo.

La Figura 18.6 muestra una implementación que cumple con la opción 4 para el cifrado extremo-a-extremo. En la figura se ha ignorado el cifrado de enlace. Ésta se puede incorporar, o no, según se requiera. Para este esquema, se identifican dos clases de claves:

- **Clave de sesión:** cuando dos sistemas finales (computadores, terminales, etc.) desean comunicarse, establecen una conexión lógica (por ejemplo, circuitos virtuales). Durante la duración de la conexión lógica, todos los datos de usuario se cifran con una clave de sesión de un solo uso. Al terminar la sesión, o conexión, la clave de sesión se destruye.
- **Clave permanente:** es la clave usada entre entidades con el objetivo de distribución de claves de sesión.

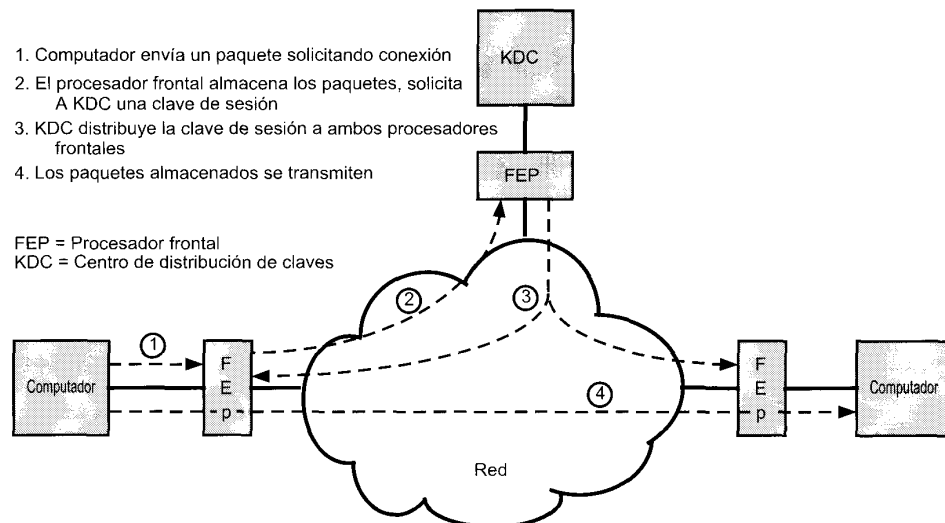


Figura 18.6. Distribución de claves automática para protocolos orientados a conexión.

La configuración consta de los siguientes elementos:

- **Centro de distribución de claves (CDC):** el centro de distribución de claves determina a qué sistemas se les permite comunicarse con otros. Cuando se concede un permiso a dos sistemas para establecer una conexión, el centro de distribución de claves proporciona una clave de sesión de un solo uso a esa conexión.
- **Procesador frontal (PF):** el procesador frontal lleva a cabo el cifrado extremo-a-extremo y obtiene las claves de sesión en representación de su computador o terminal.

Los pasos necesarios para establecer una conexión se muestran en la figura. Cuando un computador desea establecer una conexión con otro computador, transmite un paquete de solicitud de conexión (paso 1). El procesador frontal guarda el paquete y solicita permiso al CDC para establecer una conexión (paso 2). La comunicación entre el PF y el CDC está cifrada utilizando una clave maestra compartida sólo por el PF y el CDC. Si el CDC aprueba la solicitud de conexión, genera una clave de sesión y la entrega a los dos procesadores frontales apropiados, utilizando una clave única permanente para cada procesador frontal (paso 3). El procesador frontal solicitante puede ahora liberar el paquete de solicitud de conexión y se establece la conexión entre los dos sistemas finales (paso 4). Todos los datos de usuario intercambiados entre los dos sistemas finales son cifrados por sus respectivos procesadores frontales usando la clave de sesión única.

La técnica de distribución de claves automática proporciona las características de flexibilidad y de dinamismo necesarias para permitir a varios usuarios de terminales acceder a una serie de computadores, y a los computadores intercambiar datos con cada uno de los otros.

Por supuesto, el cifrado por clave pública utiliza otra técnica de distribución de claves, pero será discutida en la Sección 18.4.

RELLENO DE TRÁFICO

Se ha mencionado, en algunos casos, que los usuarios están preocupados por la seguridad en casos de análisis de tráfico. Con el uso de cifrado de enlace, las cabeceras de los paquetes se cifran, reduciendo la oportunidad de hacer un análisis de tráfico. Sin embargo, es todavía posible en estas circunstancias que un agresor calcule la cantidad de tráfico en la red y observe la cantidad de tráfico que entra y sale de cada sistema final. Una contramedida efectiva a esta agresión es el relleno de tráfico.

El relleno de tráfico es una función que produce salida de texto cifrado continuamente, incluso en ausencia de texto nativo. Se genera un flujo de datos aleatorio continuamente. Cuando hay disponible texto nativo, se cifra y se transmite. Cuando el texto nativo no está presente, los datos aleatorios se cifran y transmiten. Esto hace imposible que un agresor distinga entre flujo de datos verdaderos y ruido y por tanto le resulta imposible deducir la cantidad de tráfico.

18.3. AUTENTIFICACIÓN DE MENSAJES Y FUNCIONES DE DISPERSIÓN (HASH)

El cifrado protege de las agresiones pasivas (escuchas). Un requisito diferente es la protección a las agresiones activas (falsificación de datos y transacciones). La protección contra esas agresiones se conoce como autenticación de mensajes.

TÉCNICAS DE AUTENTIFICACIÓN DE MENSAJES

Un mensaje, fichero, documento, u otra colección de datos se dice que está autenticada cuando son genuinos y vienen del origen pretendido. La autenticación de mensajes es un procedimiento que permite a las partes que se comunican verificar que los mensajes recibidos son auténticos. Los dos aspectos importantes son verificar que el contenido del mensaje no se ha alterado y que el origen es auténtico.

También estaremos interesados en verificar que los datos son oportunos (que no han sido artificialmente retrasados y reemplazados) y verificar la secuencia relativa a otros mensajes que fluyen entre dos partes.

Autenticación utilizando cifrado convencional

Es posible llevar a cabo la autenticación simplemente mediante el uso del cifrado convencional. Si suponemos que solamente el emisor y el receptor comparten la clave (que es lo que debería ocurrir), entonces solamente el emisor genuino sería capaz de cifrar un mensaje satisfactoriamente para la otra parte. Además, si el mensaje incluye un código de detección de errores y un número de secuencia, se le asegura al receptor que no se han hecho alteraciones y que la secuencia es la adecuada. Si el mensaje también incluye una marca de tiempo, el receptor tiene la seguridad de que el mensaje no se ha retrasado más de lo normalmente esperado en el tránsito por la red.

Autenticación de mensajes sin cifrado de mensajes

En esta sección, examinamos varias técnicas para autenticar mensajes sin basarse en el cifrado. En todas estas técnicas, se genera una etiqueta de autenticación que se incorpora al mensaje para su transmisión. El mensaje mismo no está cifrado y se puede leer en el destino independientemente de la función de autenticación en el destino.

Ya que las técnicas descritas en esta sección no cifran el mensaje, no se proporciona privacidad en el mensaje. Como el cifrado convencional proporciona autenticación, y como se utiliza de una forma amplia con productos disponibles, ¿por qué no utilizar esta técnica, que proporciona el secreto y la autenticación de mensajes? En [DAVI90] se sugieren tres situaciones en las que es preferible autenticación sin tener que hacer los mensajes secretos.

1. Existen una serie de aplicaciones en las que el mensaje se difunde a varios destinos. Por ejemplo, para notificar a los usuarios que la red está disponible o una señal de alarma en un centro de control. Es más barato y más seguro tener solamente un destino para monitorizar la autenticación. Así, el mensaje que se va a difundir es texto nativo con una etiqueta de mensaje de autenticación asociada. El sistema responsable lleva a cabo la autenticación. Si ocurre una violación, se alertan a los otros sistemas destinos mediante una alarma general.
2. Otro posible escenario es un intercambio en el que una de las partes tiene una carga muy elevada y no tiene tiempo de descifrar los mensajes que llegan. La autenticación se realiza de una forma selectiva, eligiendo los mensajes de forma aleatoria para realizar comprobaciones.
3. La autenticación de un programa de computador en texto nativo es un servicio interesante. El programa se puede ejecutar sin tener que descifrarlo cada vez que se ejecuta, ya que de lo contrario se produciría un derroche de los recursos del procesador. Sin embargo, si la etiqueta de autenticación de mensaje fuera incorporada al programa, se podría comprobar siempre y cuando se necesite tener certeza de la integridad del programa.

Así, existen ámbitos distintos para cifrado y autenticación, dentro de los requisitos de seguridad.

Código de autenticación de mensajes (CAM)

Otra técnica de autenticación supone el uso de una clave secreta que genera un pequeño bloque de datos, conocido como código de autenticación de mensaje, y que se incorpora al mensaje. Esta técnica supone que dos partes comunicantes, digamos A y B, comparten una clave secreta común K_{AB} . Cuando A tiene un mensaje que enviar a B, calcula el código de autenticación del mensaje como una función del mensaje y la clave: $MAC_M = F(K_{AB}, M)$. Luego se transmite el mensaje y el código al destino. El receptor realiza los mismos cálculos en el mensaje recibido, utilizando la misma clave secreta, para generar un nuevo código de autenticación de mensaje. El código recibido se compara con el código cal-

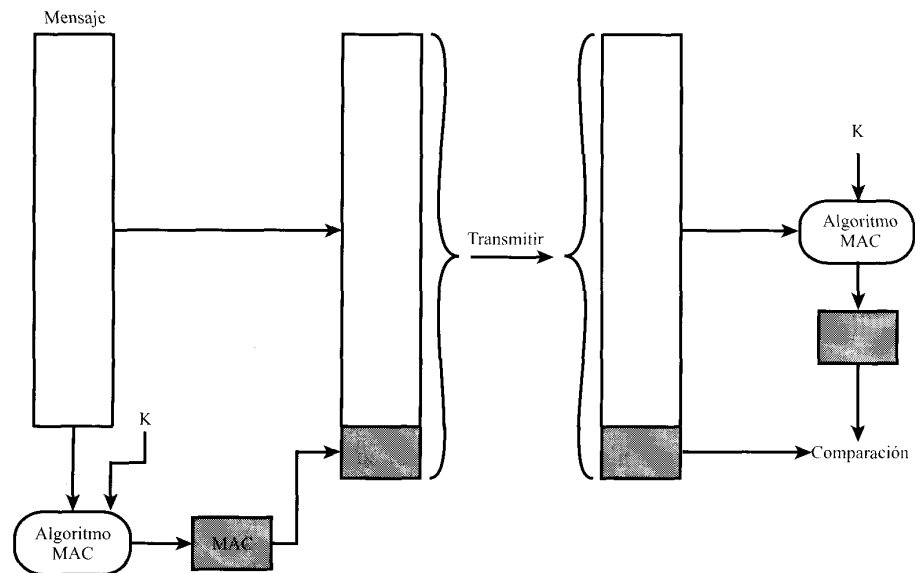


Figura 18.7. Autenticación de mensajes utilizando un código de autenticación de mensaje (MAC).

culado (Figura 18.7). Si suponemos que sólo el emisor y el receptor conocen la identidad de la clave, y si el código recibido coincide con el calculado, entonces:

1. El receptor está seguro que el mensaje no ha sido alterado. Si un agresor altera el mensaje pero no el código, el código calculado en el receptor diferirá del código recibido. Ya que se supone que el agresor no conoce la clave secreta, éste no podrá modificar el código para que corresponda con la modificación en el mensaje.
2. El receptor está seguro que el mensaje es del emisor pretendido. Ya que nadie más conoce la clave secreta, nadie podría preparar un mensaje con el código apropiado.
3. Si el mensaje incluye un número de secuencia (como los que se utilizan con X.25, HDLC y TCP), entonces el receptor puede estar seguro de la secuencia adecuada, ya que un agresor no puede alterar el número de secuencia satisfactoriamente.

Se pueden utilizar una serie de algoritmos para generar el código. El Buró Nacional de Estándares, en su publicación *Modos de operación de DES*, recomienda el uso del algoritmo DEA. El algoritmo DEA se utiliza para generar una versión cifrada del mensaje, y los últimos bits del texto cifrado se utilizan como código. Son normales códigos de 16 o 32 bits.

El proceso descrito es similar al de cifrado. Una diferencia es que el algoritmo de autenticación no necesita ser reversible, como es el caso para el descifrado. Resulta que debido a las propiedades matemáticas de la función de autenticación, éste es menos vulnerable que el cifrado cuando se rompe el mensaje.

Función de dispersión de un solo sentido

Una variación al código de autenticación de mensajes y que ha recibido mucha atención recientemente es la función de dispersión de un solo sentido. Como con el código de autenticación de mensajes, una función de dispersión acepta un mensaje de longitud variable M como entrada y produce una etiqueta de

tamaño fijo $H(M)$, como salida. A diferencia del CAM, la función de dispersión no tiene como entrada una clave secreta. El resumen del mensaje se envía junto al mensaje de tal forma que el resumen del mensaje se puede utilizar en el receptor para autentificarlo.

La Figura 18.8 muestra tres formas en las que se puede autentificar un mensaje. El resumen del mensaje se puede cifrar utilizando cifrado convencional (parte (a)); si se supone que sólo el emisor y el receptor comparten la clave, se asegura la autentificación. El mensaje también se puede cifrar utilizando cifrado por clave pública (parte (b)); esto se explica en la Sección 18.4. La técnica de clave pública tiene dos ventajas: proporciona una firma digital así como autentificación de mensajes, y no requiere la distribución de claves a las partes que se comunican.

Estas dos técnicas tienen la ventaja sobre otras técnicas que cifran el mensaje entero en que requieren menos computación. Sin embargo, existe una serie de razones para desarrollar una técnica que evite el cifrado del conjunto. En [TSUD92] se indican algunas de estas razones:

- El software de cifrado es muy lento. Incluso aunque la cantidad de datos que se van a cifrar sea pequeña, puede producirse una secuencia constante de mensajes entrando y saliendo del sistema.

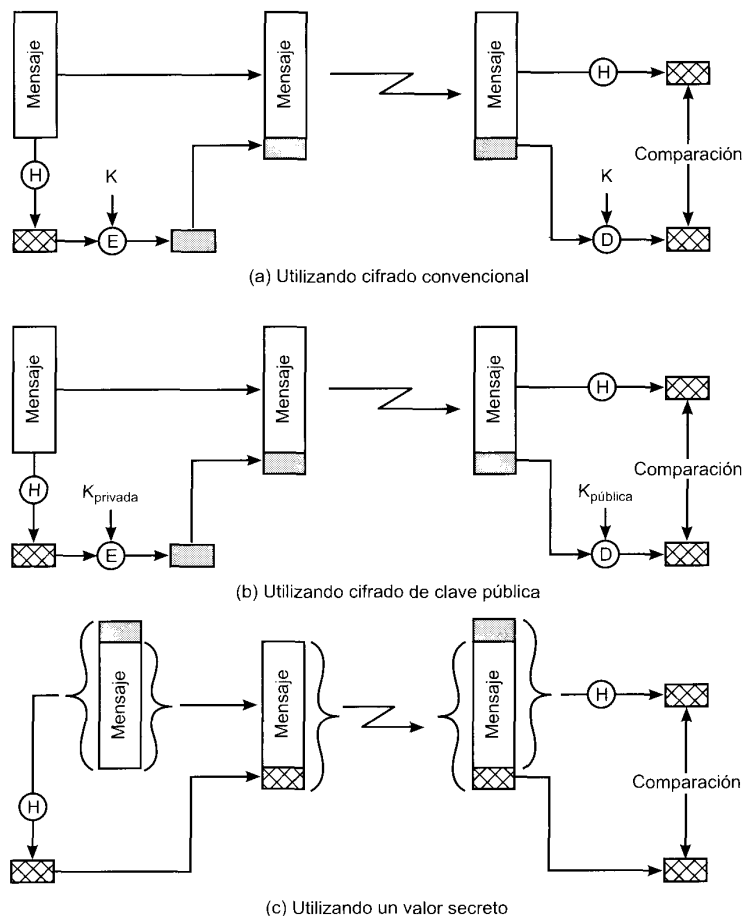


Figura 18.8. Autentificación de mensajes utilizando una función de dispersión de un solo sentido.

- El coste del hardware de cifrado no es despreciable. Están disponibles implementaciones en circuitos de bajo coste del algoritmo DES, pero este coste se puede incrementar bastante si todos los nodos de una red tienen que tener esta capacidad.
- El hardware de cifrado está optimizado para tamaños de datos grandes. Para bloques de datos pequeños, se emplea una gran cantidad de tiempo en la inicialización y en la invocación.
- Los algoritmos de cifrado pueden estar protegidos por patentes. Algunos algoritmos de cifrado, como el algoritmo de clave pública RSA, están patentados y se necesita licencia, lo que aumenta el coste.
- Los algoritmos de cifrado pueden estar sujetos a control de exportación. Esto es cierto para DES.

La Figura 18.8c muestra una técnica que utiliza una función de dispersión pero no utiliza cifrado para realizar la autenticación del mensaje. Esta técnica supone que las dos partes comunicantes, digamos A y B, comparten un valor secreto común S_{AB} . Cuando A envía un mensaje a B, calcula la función de dispersión utilizando la concatenación de la clave secreta y el mensaje: $MD_M = H(S_{AB}||M)$ ¹. Entonces envía $[M||MD_M]$ a B. Ya que B posee S_{AB} , puede recalcular $H(S_{AB}||M)$ y verificar MD_M . Ya que el valor secreto no se envía, no es posible que un agresor modifique un mensaje interceptado. Mientras el valor secreto permanezca oculto, no es posible que un agresor genere un mensaje falso.

Esta tercera técnica, consistente en utilizar un valor secreto común, es la que ha adoptado IP para implementar las características en seguridad; también ha sido especificada para SNMPv3, discutido en el Capítulo 19.

FUNCIONES DE DISPERSIÓN SEGURAS

La función de dispersión de un solo sentido, o función de dispersión segura, es importante no sólo para la autenticación de mensajes, sino para las firmas digitales. En esta sección comenzaremos discutiendo los requisitos necesarios para obtener una función de dispersión segura. Después examinaremos una de las funciones de dispersión más relevantes, SHA-1.

Requisitos de una función de dispersión segura

El objetivo de una función de dispersión es producir una «huella dactilar» de un fichero, mensaje u otro bloque de datos. Para que sea útil para la autenticación, una función de dispersión H debe tener las siguientes propiedades:

1. H puede ser aplicada a bloques de datos de cualquier tamaño.
2. H produce una salida de longitud fija.
3. Para un x dado es relativamente fácil calcular $H(x)$, haciendo práctica la implementación hardware y software.
4. Para un código dado h , es imposible computacionalmente encontrar un x tal que $H(x) = h$.
5. Para un bloque dado x , es imposible computacionalmente encontrar un $y \neq x$ con $H(y) = H(x)$.
6. Es imposible computacionalmente encontrar una pareja (x, y) tal que $H(x) = H(y)$.

Las tres primeras propiedades son requisitos para la aplicación práctica de una función de dispersión a la autenticación de mensajes.

La cuarta propiedad es la propiedad de «un solo sentido»: es fácil generar un código dado un mensaje, pero virtualmente imposible generar un mensaje dado un código. Esta propiedad es importante si la técnica de autenticación supone el uso de un valor secreto (Figura 18.8c). El valor secreto no se envía;

¹ $||$ indica concatenación.

sin embargo, si la función de dispersión no es de un solo sentido, un agresor puede descubrir fácilmente el valor secreto: si el agresor puede observar o interceptar una transmisión, el agresor obtiene el mensaje M y el código de dispersión $MD_M = H(S_{AB}||M)$. El agresor entonces invierte la función de dispersión para obtener $S_{AB}||M = H^{-1}(MD_M)$. Ya que el agresor tiene ahora M y $S_{AB}||M$ es una cuestión trivial obtener S_{AB} .

La quinta propiedad garantiza que no se puede encontrar un mensaje alternativo que produzca el mismo valor que un mensaje dado. Esto previene la falsificación cuando se utiliza un código de dispersión cifrado (Figuras 18.8a y 18.8b). Si esta propiedad no fuera válida, un agresor sería capaz de realizar la secuencia siguiente: primero, observar o interceptar un mensaje más su código de dispersión cifrado; segundo, generar un código de dispersión descifrado a partir del mensaje; tercero, generar un mensaje alternativo con el mismo código de dispersión.

Una función de dispersión que satisface las cinco primeras propiedades de la lista anterior se le conoce como función de dispersión débil. Si satisface también la sexta propiedad entonces se le conoce como función de dispersión fuerte. La sexta propiedad protege de una clase de agresión sofisticada conocida como agresión de cumpleaños².

Además de proporcionar autenticación, un resumen del mensaje proporciona también integridad de los datos. Lleva a cabo la misma función que la secuencia de comprobación de trama: si algún bit se altera accidentalmente en el tránsito, el resumen del mensaje producirá un error.

LA FUNCIÓN DE DISPERSIÓN SEGURA SHA-1

El algoritmo seguro de dispersión (SHA) fue desarrollado por el Instituto Nacional de Estándares y Tecnología (NIST) y publicado como estándar federal para el procesamiento de la información (FIPS PUB 180) en 1993; en 1985 se publicó una versión revisada como FIPS PUB 180-1 y generalmente se conoce como SHA-1.

El algoritmo toma como entrada un mensaje con una longitud máxima de 2^{64} bits y produce un resumen del mensaje de 160 bits. La entrada se procesa en bloques de 512 bits. La Figura 18.9 muestra el procesamiento general de un mensaje para producir un resumen. El procesamiento consta de los siguientes pasos:

Paso 1: incorporar bits de relleno. Se incorporan al mensaje bits de relleno para que su longitud en bits sea congruente con 448 módulo 512 (longitud = 448 modo 512). Esto es, la longitud del mensaje al que se le incorpora el relleno es 64 bits menos que un múltiplo entero de 512 bits. Siempre se incorpora el relleno, incluso si el mensaje tiene ya la longitud deseada. Así, el número de bits de relleno está en el rango de 1 a 512. El relleno consiste de un único bit con valor 1 seguido de los bits necesarios con valor cero.

Paso 2: incorporar la longitud. Se incorpora un bloque de 64 bits al mensaje. Este bloque se trata como un entero de 64 bits sin signo (los bits más significativos primero) y contiene la longitud del mensaje original (antes de incorporar el relleno). La inclusión de un valor de longitud hace más difícil un tipo de agresión, conocida como agresión de relleno [TSUD92].

La salida de los dos primeros pasos produce un mensaje que es un múltiplo entero de 512 bits en longitud. En la Figura 18.9, el mensaje ampliado se representa como una secuencia de bloques de 512 bits $Y_0, Y_1, \dots, Y_{L-1}$, de manera que la longitud del mensaje ampliado es $L \times 512$ bits. Equivalentemente, el resultado es un múltiplo de 16 palabras de 32 bits. Las palabras del mensaje resultante se denotan como $M[0 \dots N-1]$, con N un múltiplo entero de 16. Así, $N = L \times 16$.

Paso 3: inicializar la memoria temporal MD. Se utiliza una memoria temporal de 160 bits para almacenar los resultados intermedios y finales de la función de dispersión. La memoria temporal se

² Véase [STAL99a] para una discusión del ataque de cumpleaños.

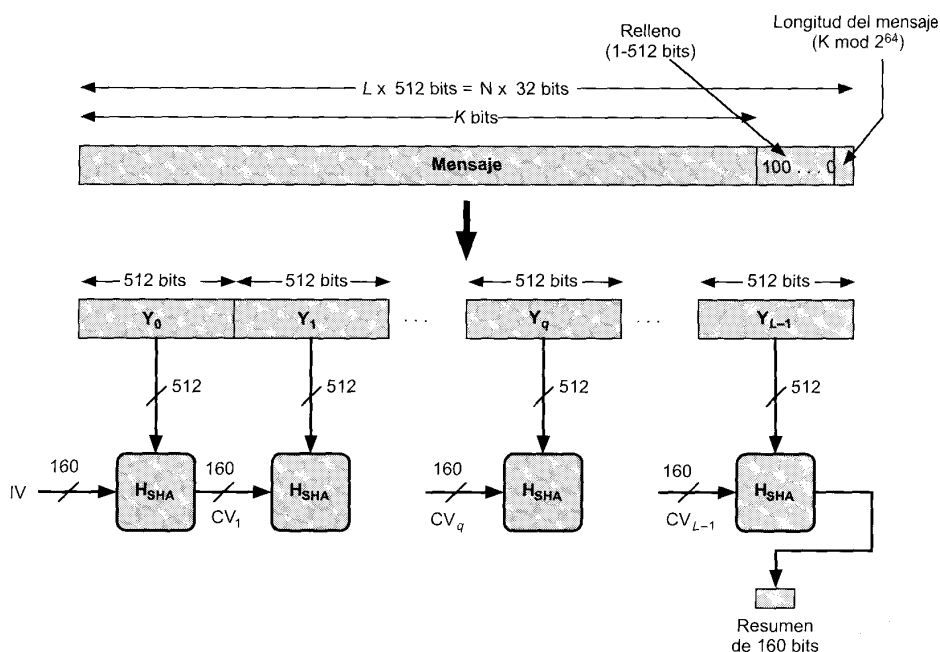


Figura 18.9. Generación de resumen de mensaje utilizando SHA-1.

puede representar con 5 registros de 32 bits (A, B, C, D, E). Estos registros se inicializan a los valores enteros de 32 bits siguientes (valores hexadecimales):

A = 67452301
 B = EFCDAB89
 C = 98BADCFE
 D = 10325476
 E = C3D2E1F0

Paso 4: procesar el mensaje en bloques de 512 bits (palabras de 16 bits). El corazón del algoritmo es un módulo que consta de 4 rondas de procesamiento de 20 pasos cada uno. La lógica asociada se muestra en la Figura 18.10. Las cuatro rondas tienen una estructura similar pero cada una utiliza una función lógica primitiva diferente, conocidas como f_1 , f_2 , f_3 y f_4 .

Cada ronda toma como entrada el bloque de 512 bits que se está procesando (Y_q) y el valor de la memoria temporal de 160 bits ABCDE y actualiza el contenido de la memoria temporal. Cada ronda también hace uso de una constante aditiva K_t , donde $0 \leq t \leq 79$ indica uno de los 80 pasos a lo largo de las cinco rondas. De hecho, sólo se utilizan cuatro constantes diferentes. Los valores, en hexadecimal y decimal, son los siguientes:

Número de paso	Hexadecimal	Tomar la parte entera de:
$0 \leq t \leq 19$	$K_t = 5A827999$	$[2^{30} \times \sqrt{2}]$
$20 \leq t \leq 39$	$K_t = 6ED9EBA1$	$[2^{30} \times \sqrt{3}]$
$40 \leq t \leq 59$	$K_t = 8F1BBCDC$	$[2^{30} \times \sqrt{5}]$
$60 \leq t \leq 79$	$K_t = CA62C1D6$	$[2^{30} \times \sqrt{10}]$

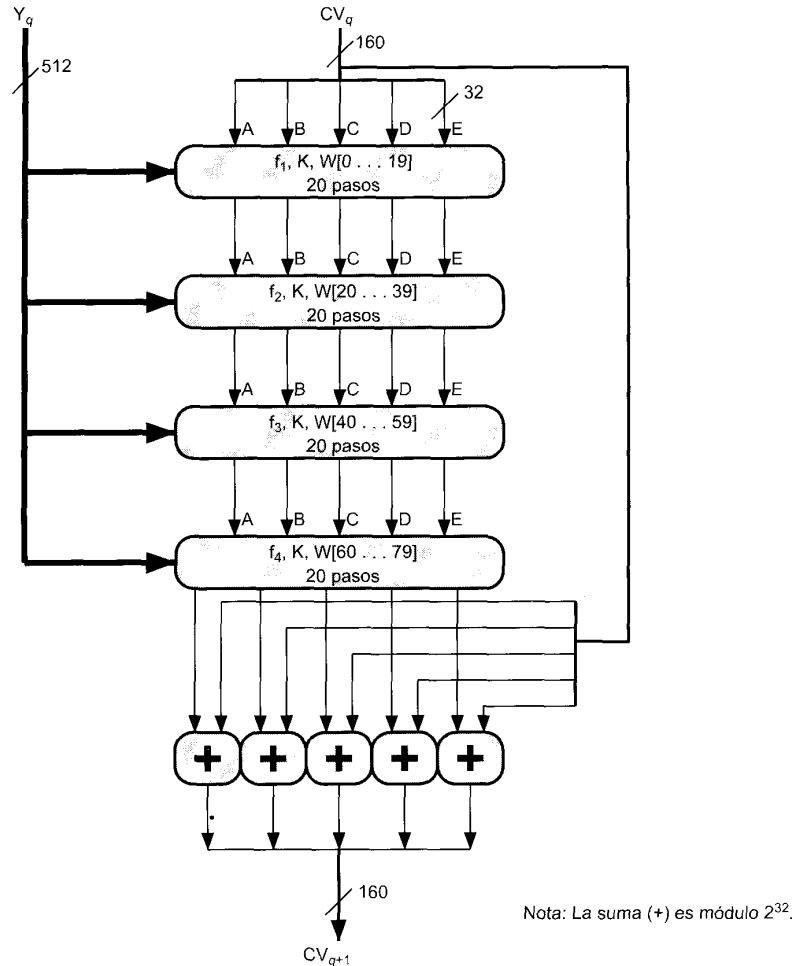


Figura 18.10. Procesamiento SHA-1 de un único bloque de 512 bits.

La salida de la ronda cuarta (octavo paso) se suma a la entrada de la primera ronda (CV_q) para producir (CV_{q+1}). Esta suma se hace independientemente para cada una de las cinco palabras almacenadas en la memoria temporal con cada palabra correspondiente en CV_q , utilizando suma módulo 2^{32} .

Paso 5: salida. Después de que los L bloques de 512 bits se han procesado, el resumen del mensaje de 160 bits es la salida de la etapa L -ésima.

El algoritmo SHA-1 tiene la propiedad de que cada bit del código de dispersión es una función de cada bit de la entrada. La repetición compleja de la función base f_i produce resultados que están bien mezclados; esto es, no es probable que dos mensajes elegidos aleatoriamente, incluso si exhiben regularidades similares, tengan el mismo código de dispersión. A menos que exista alguna debilidad oculta en SHA-1, lo cual todavía no se ha publicado, la dificultad de alcanzar dos mensajes que tienen el mismo resumen de mensaje es del orden de 2^{80} operaciones, mientras que la dificultad de encontrar un mensaje con un resumen dado es del orden de 2^{160} operaciones.

18.4. CIFRADO DE CLAVE PÚBLICA Y FIRMAS DIGITALES

El cifrado de clave pública tiene la misma importancia que el cifrado convencional, y encuentra su utilización en la autenticación de mensajes y en la distribución de claves. Esta sección examina primero los conceptos básicos del cifrado de clave pública, seguido por una discusión de las firmas digitales. Después se discute el algoritmo de clave pública más utilizado: RSA. Después se examina el problema de la distribución de claves.

CIFRADO DE CLAVE PÚBLICA

El cifrado de clave pública, propuesta en público por primera vez por Diffie y Hellman en 1976 [DIFF76], es el primer avance realmente revolucionario en el cifrado en literalmente miles de años. Y esto es debido a una razón, el algoritmo de clave pública se basa en funciones matemáticas en lugar de sustituciones y permutaciones. Pero lo más importante, la criptografía de clave pública es asimétrica y supone el uso de dos claves independientes, en contraste con el cifrado simétrico convencional, que sólo utiliza una clave. Utilizar dos claves tiene consecuencias profundas en las áreas de privacidad, distribución de claves y autenticación.

Antes de proseguir, deberíamos mencionar varios malentendidos comunes que afectan al cifrado de clave pública. Uno de tales malentendidos es que el cifrado de clave pública es más seguro frente al criptoanálisis que el cifrado convencional. En realidad, la seguridad de cualquier esquema de cifrado depende de (1) la longitud de la clave y (2) el trabajo computacional que requiere romper un cifrado. No hay nada en principio del cifrado convencional o de clave pública que haga a uno superior al otro desde el punto de vista de resistir el criptoanálisis. Un segundo malentendido es que el cifrado de clave pública es una técnica de uso general que ha hecho que se quede obsoleta el cifrado convencional. Por el contrario, a causa de la computación suplementaria de los esquemas actuales de cifrado por clave pública, no es probable que el cifrado convencional sea abandonado. Finalmente, existe el parecer de que la distribución de claves es trivial cuando se utiliza cifrado de clave pública, comparado con el diálogo más bien molesto que se requiere con los centros de distribución de claves en el cifrado convencional. De hecho, se necesita alguna forma de protocolo, a menudo implicando a un agente central, y los procedimientos implicados no son más sencillos o más eficientes que los que se requieren para el cifrado convencional.

El esquema de cifrado de clave pública tiene seis ingredientes (Figura 18.11):

- **Texto nativo:** es el mensaje legible o datos que se pasan al algoritmo como entrada.
- **Algoritmo de cifrado:** el algoritmo de cifrado lleva a cabo varias operaciones sobre el texto nativo.
- **Clave pública y privada:** éste es el par de claves que se ha seleccionado para que una se utilice para el cifrado y la otra para el descifrado. Las transformaciones exactas que realiza el algoritmo de cifrado dependen de la clave pública o privada que se suministra como entrada.
- **Texto cifrado:** es el mensaje mezclado producido como salida. Depende del texto nativo y de la clave. Para un mensaje dado, dos claves diferentes producen dos textos cifrados diferentes.
- **Algoritmo de descifrado:** este algoritmo acepta el texto cifrado y la otra clave y produce el texto nativo original.

Como el nombre sugiere, la clave pública del par se hace pública para que la utilice el resto de la gente, mientras que la clave privada solamente la conoce el dueño. Un algoritmo de criptografía de clave pública de propósito general se basa en una clave de cifrado y en una clave diferente, pero relacionada, para el descifrado. Además, estos algoritmos tienen las siguientes características importantes:

- No es factible computacionalmente determinar la clave de descifrado dado solamente el algoritmo de criptografía y la llave de cifrado.

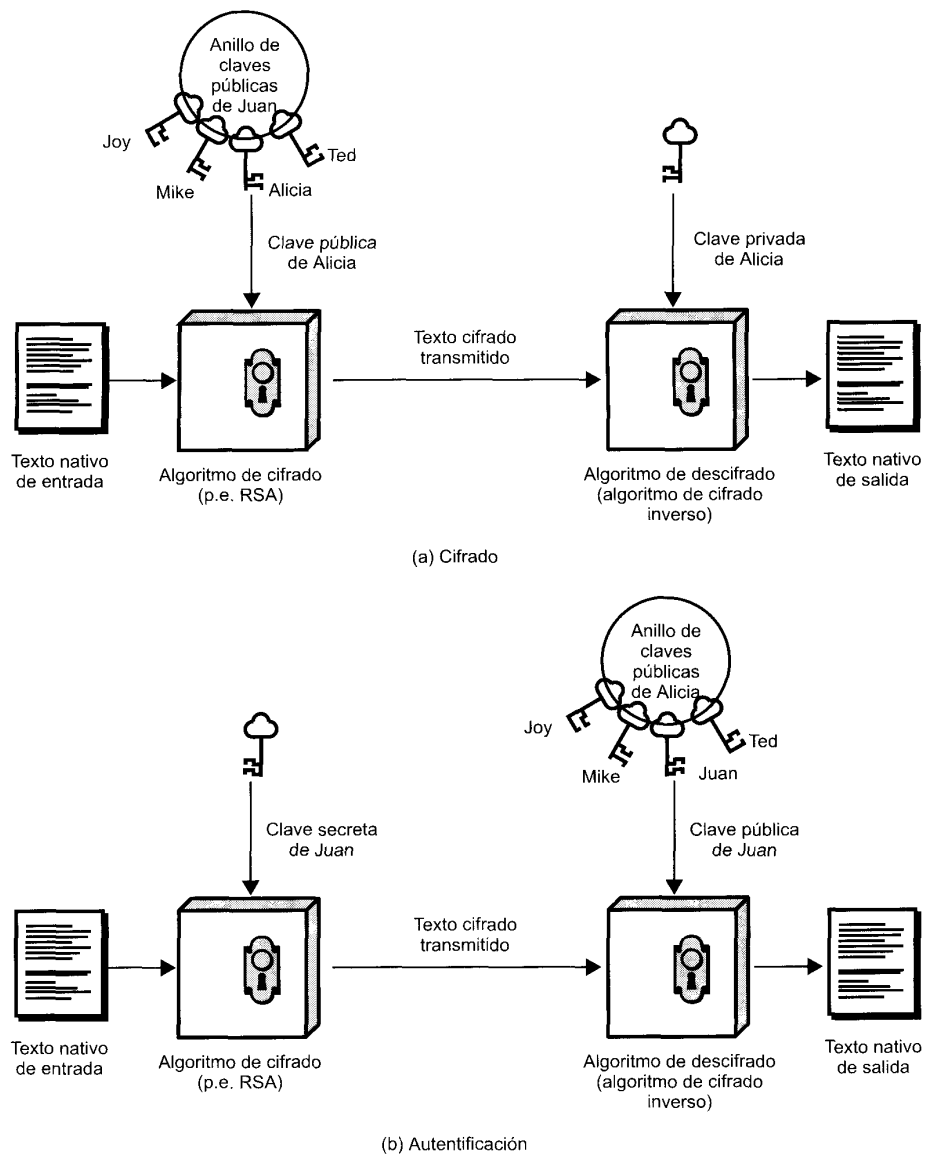


Figura 18.11. Cifrado de clave pública.

- Para la mayoría de los esquemas de clave pública, cualquiera de las dos claves que se utilizan, se puede emplear para el cifrado y la otra para el descifrado.

Los pasos esenciales son los siguientes:

1. Cada usuario genera un par de claves que se van a utilizar para el cifrado y el descifrado de los mensajes.

2. Cada usuario publica una de las dos claves de cifrado situándola en un registro o fichero público. Ésta es la clave pública. La clave compañera se mantiene privada. Como sugiere la Figura 18.11, cada usuario mantiene una colección de claves públicas de otros usuarios.
3. Si Juan desea enviar un mensaje privado a Alicia, él cifra el mensaje utilizando la clave pública de Alicia.
4. Cuando Alicia recibe el mensaje, lo descifra utilizando su clave privada. Ningún otro destino puede descifrar el mensaje ya que solamente Alicia conoce la clave privada de Alicia.

Con esta técnica, todos los participantes tienen acceso a las claves públicas y las claves privadas se generan localmente por cada participante y por lo tanto nunca se distribuyen. Mientras un usuario controle su clave privada, los mensajes que le llegan son seguros. Un usuario puede cambiar su clave privada en cualquier instante de tiempo y publicar la clave pública compañera para reemplazar la clave pública obsoleta.

FIRMAS DIGITALES

El cifrado de clave pública se puede utilizar de otra forma, como se muestra en la Figura 18.11b. Suponga que Juan quiere enviar un mensaje a Alicia y, aunque no es importante que el mensaje se mantenga secreto, quiere que Alicia esté segura que en realidad el mensaje es de él. En este caso, Juan utiliza su propia clave privada. Cuando Alicia recibe el texto cifrado, encuentra que puede descifrarlo con la clave pública de Juan, demostrando así que el mensaje ha sido cifrado por Juan. Nadie más tiene la clave privada de Juan y, por lo tanto, nadie más ha podido crear el texto cifrado que se descifra con la clave pública de Juan. De esta forma, el mensaje cifrado completo sirve como **firma digital**. Además, es imposible alterar el mensaje sin acceder a la clave privada de Juan, por lo tanto el mensaje está autenticado en términos de la fuente y de integridad de los datos.

En el esquema precedente, se cifra el mensaje entero, lo que, aunque valida al autor y al contenido, requiere una gran cantidad de almacenamiento. Cada documento debe guardarse en texto nativo para ser útil por motivos prácticos. Se debe guardar también una copia del texto cifrado para que se pueda verificar el origen y el contenido en caso de disputa. Una forma más eficiente de conseguir el mismo resultado es cifrar un bloque pequeño de bits que sea una función del documento. Este bloque, llamado autenticador, debe tener la propiedad de que no sea factible cambiar el documento sin cambiar el autenticador. Si el autenticador se cifra con la clave privada del emisor, sirve como una firma que verifica al origen, el contenido y el secuenciamiento. Un código seguro de dispersión como es SHA-1 puede servir como función.

Es importante enfatizar que el proceso de cifrado que se acaba de describir no proporciona privacidad. Esto es, el mensaje que se envía está seguro frente a alteraciones, pero no lo está de ser escuchado. Esto es obvio en el caso de una firma basada en una parte del mensaje, ya que el resto del mensaje no está cifrado. Incluso en el caso de cifrar el mensaje entero, no hay protección de confidencialidad ya que cualquier observador puede descifrar el mensaje utilizando la clave pública del emisor.

EL ALGORITMO DE CIFRADO DE CLAVE PÚBLICA RSA

Uno de los primeros esquemas de clave pública fue desarrollado en 1977 por Ron Rivest, Adi Shamir y Len Adleman en el MIT y publicado por primera vez en 1978 [RIVE78]. El esquema RSA ha sido considerado desde entonces como la única técnica mundialmente aceptada e implementada de algoritmo de cifrado de clave pública. RSA es un cifrador de bloque en el que el texto nativo y el texto cifrado son enteros entre 0 y $n - 1$ para algún n .

Para algún texto nativo M y un bloque cifrado C , el cifrado y el descifrado son de la siguiente forma:

$$C = M^e \bmod n$$

$$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$$

Ambos, el emisor y el receptor deben conocer el valor de n . El emisor conoce el valor de e y el receptor sólo debe conocer el valor de d . De esta forma, éste es un algoritmo de clave pública con una clave pública dada por $KU = [e, n]$ y una clave privada $KR = [d, n]$. Para que este algoritmo sea satisfactoriamente un algoritmo de cifrado de clave pública, se deben satisfacer los siguientes requisitos:

1. Es posible encontrar valores de e, d, n tal que $M^{ed} = M \bmod n$ para todo $M < n$.
2. Es relativamente fácil calcular M^e y C^d para todos los valores de $M < n$.
3. Es imposible determinar d dado e y n .

Los dos primeros requisitos son fáciles de satisfacer. El tercer requisito se puede satisfacer para un valor grande de e y n .

La Figura 18.12 resume el algoritmo RSA. Se empieza por seleccionar dos números primos, p y q y calculando su producto n , que es el módulo para el cifrado y el descifrado. A continuación, necesitamos la cantidad $\phi(n)$, que se conoce como totalizador (*totient*) de Euler de n , y que es el número de enteros positivos menores que n y relativamente primo a n . Entonces, se selecciona el entero e que es relativamente primo a $\phi(n)$, esto es, el máximo común divisor de e y $\phi(n)$ debe ser 1. Finalmente, se calcula d como la inversa del multiplicador de e , módulo $\phi(n)$. Se puede demostrar que d y e tienen las propiedades deseadas.

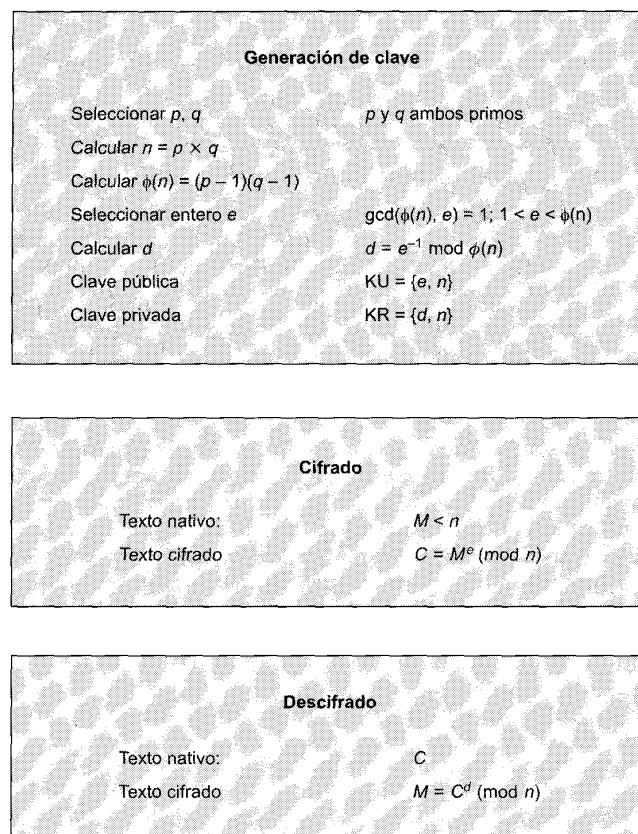


Figura 18.12. El algoritmo RSA.

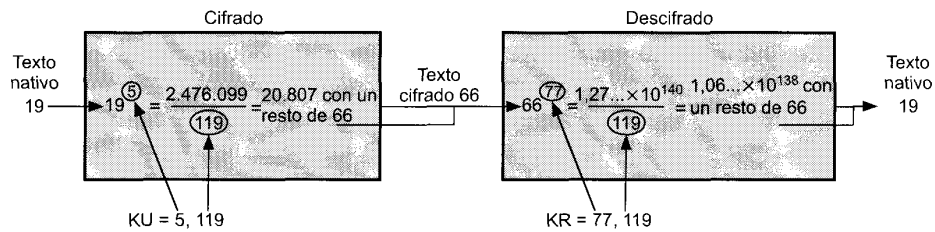


Figura 18.13. Ejemplo del algoritmo RSA.

Suponga que un usuario A ha publicado su clave pública y que el usuario B quiere enviar un mensaje M a A. Para ello, B calcula $C = M^e \pmod{n}$ y transmite C . Cuando se recibe el mensaje cifrado, A lo descifra calculando $M = C^d \pmod{n}$.

En la Figura 18.13 se muestra un ejemplo. En este ejemplo, las claves se generaron como sigue:

1. Seleccionar dos números primos, $p = 7$ y $q = 17$.
2. Calcular $n = pq = 7 \times 17 = 119$.
3. Calcular $\phi(n) = (p - 1)(q - 1) = 96$.
4. Seleccionar e tal que e es relativamente primo a $\phi(n) = 96$ y es menor que $\phi(n)$; en este caso, $e = 5$.
5. Determinar d tal que $de = 1 \pmod{96}$ y $d < 96$. El valor correcto es $d = 77$, ya que $77 \times 5 = 385 = 4 \times 96 + 1$.

Las claves resultantes son la clave pública $KU = \{5, 119\}$ y la clave privada $KR = \{77, 119\}$. El ejemplo muestra el uso de estas claves para un texto nativo de entrada $M = 19$. En el cifrado, 19 se eleva a la quinta potencia, produciendo 2.476.099. Dividiendo por 119, se obtiene un resto de 66. Por lo tanto $19^5 \equiv 66 \pmod{119}$, y el texto cifrado es 66. En el descifrado, se determina que $66^{77} \equiv 19 \pmod{119}$.

Existen dos técnicas posibles para inutilizar el algoritmo RSA. La primera es una técnica de fuerza bruta: probar todas las claves privadas posibles. Así, cuanto más grandes sean en bits los números e y d , el algoritmo será más seguro. Sin embargo, a causa de los cálculos necesarios, tanto la generación de claves como el cifrado/descifrado son complejos y cuanto más grandes sean los tamaños de las claves más lento será el sistema.

La mayoría de las discusiones sobre el criptoanálisis se han centrado en la tarea de factorizar n en sus dos números primos. Para un n grande con factores primos grandes, la factorización es un problema difícil, pero no tanto como solía ser. Una ilustración sorprendente de esto es la siguiente. En 1977, los tres inventores de RSA desafiaron a los lectores de *Scientific American* a decodificar un texto cifrado que imprimieron en la columna «Juegos Matemáticos» de Martín Gardner. Ofrecieron 100 dólares como recompensa a quien les enviara la oración en texto nativo, un hecho que ellos predijeron que no ocurriría en 40 trillones de años. En abril de 1994 un equipo mundial cooperando con Internet y utilizando 1.600 computadores ha reclamado el premio después de sólo 8 meses de trabajo [LEUT94]. Este reto utilizaba un tamaño de clave pública (longitud de n) de 129 dígitos decimales, alrededor de 428 bits. Este resultado no invalida el uso de RSA; simplemente significa que se deben utilizar tamaños de clave más grandes. Actualmente, se considera bastante potente un tamaño de clave de 1.024 bits (aproximadamente 300 dígitos decimales) para virtualmente todas las aplicaciones.

GESTIÓN DE CLAVES

Con el cifrado convencional, un requisito fundamental para dos entes que se comunican de una forma segura es que compartan una clave secreta. Supóngase que Juan quiere crear una aplicación para enviar

mensajes que le permitirá intercambiar correo electrónico de una forma segura con cualquiera que tenga acceso a Internet o a alguna otra red que los dos comparten. Supóngase que Juan quiere hacer esto utilizando sólo cifrado convencional. Con el cifrado convencional, Juan y su corresponsal, digamos, Alicia, deben presentar una forma para compartir la clave secreta única y que nadie más conoce. ¿Cómo van a hacer esto? Si Alicia está en la habitación de al lado, Juan puede generar la clave y escribirla en un papel o almacenarla en un disquete y dársela a Alicia. Pero si Alicia está en la otra parte del continente o del mundo, ¿qué puede hacer Juan? Bien, podría cifrar la clave utilizando cifrado convencional y enviarla por correo electrónico a Alicia, pero esto significa que Juan y Alicia deben compartir una clave secreta para poder cifrar esta nueva clave secreta. Por lo tanto, Juan y cualquiera que utilice este nuevo paquete de correo electrónico se debe enfrentar al mismo problema con cualquier corresponsal potencial. Cada par de corresponsales deben compartir una clave secreta única.

El mayor problema para utilizar cifrado convencional es cómo distribuir las claves secretas de una forma segura. Este problema se elimina con el cifrado de clave pública por el simple hecho de que la clave privada nunca se distribuye. Si Juan quiere establecer correspondencia con Alicia u otra persona, genera un único par de claves, una privada y otra pública. Guarda la clave privada de una forma segura y difunde la clave pública a todos y cada uno. Si Alicia hace lo mismo, entonces Juan tiene la clave pública de Alicia, Alicia tiene la clave pública de Juan y ya pueden comunicarse con seguridad. Cuando Juan desea comunicarse con Alicia, puede hacer lo siguiente:

1. Preparar un mensaje.
2. Cifrar el mensaje utilizando cifrado convencional con una clave de sesión convencional.
3. Cifrar la clave de sesión utilizando cifrado de clave pública con la clave pública de Alicia.
4. Incorporar la clave de sesión cifrada al mensaje y enviarlo a Alicia.

Solamente Alicia es capaz de descifrar la clave de sesión y por tanto de recuperar el mensaje original.

Es justo señalar, sin embargo, que hemos reemplazado un problema por otro. La clave privada de Alicia es segura ya que no necesita revelarla nunca; sin embargo, Juan debe estar segura que la clave pública con el nombre de Alicia escrita en ella es de hecho la clave pública de Alicia. Alguien podría haber difundido una clave pública y haber dicho que era la de Alicia. La forma común de solucionar este problema es ingeniosa: utilizar cifrado de clave pública para autenticar claves públicas. Esto supone la existencia de alguna autoridad o individuo señalado y de confianza que actúe como sigue:

1. Alicia genera una clave pública y la envía a la Agencia X para certificarla.
2. X determina por algún procedimiento, como con una entrevista personal, que ésta es la auténtica clave pública de Alicia.
3. X incorpora una marca de tiempo a la clave pública, genera un código de dispersión al resultado y cifra el resultado con su clave privada formando una firma.
4. La firma se incorpora a la clave pública.

Cualquiera equipado con una copia de la clave pública de X puede ahora verificar que la clave pública de Alicia es auténtica.

18.5. SEGURIDAD CON IPv4 E IPv6

En 1994, el Comité de Arquitectura de Internet (IAB, Internet Architecture Board) publicó un informe titulado *Seguridad en la Arquitectura de Internet* (RFC 1636). El informe afirmaba el consenso general de que Internet necesitaba más y una mejor seguridad e identificaba las áreas clave para los mecanismos de seguridad. Entre éstos estaba la necesidad de hacer segura la infraestructura de red para evitar la monitorización no autorizada y el control del tráfico de red y la necesidad hacer seguro el tráfico usuario final a usuario final utilizando mecanismos de autenticación y cifrado.

Estas preocupaciones están plenamente justificadas. Como confirmación, el informe anual de 1998 del Equipo de Respuesta a Emergencias en Computadores (CERT, Computer Emergency Response Team) indicaba alrededor de 1.300 incidentes de seguridad documentados afectando a casi 20.000 sitios [CERT99]. Los tipos de ataques más serios incluían «falsos IP», el que un intruso crea un paquete con una dirección IP falsa y explota las aplicaciones que utilizan la autenticación basada en direcciones IP; y varias formas de escuchas y capturas de paquetes, en los que los atacantes leen la información transmitida, incluyendo información de conexión a sistemas y contenidos de bases de datos.

En respuesta a estas cuestiones, el IAB incluyó la autenticación y el cifrado como características de seguridad necesarias en el protocolo IP de nueva generación, que se ha denominado IPv6. Afortunadamente, estas capacidades de seguridad se diseñaron para que fueran utilizadas con IPv4 e IPv6. Esto significa que los fabricantes pueden ya empezar a ofrecer estas características, y ya muchos fabricantes tienen algunas capacidades de IPSec en sus productos.

APLICACIONES DE IPSec

IPSec proporciona la capacidad de hacer segura las comunicaciones a través de una LAN, una WAN privada o pública y a través de Internet. Algunos ejemplos de su uso incluyen los siguientes:

- **Conectividad segura entre oficinas sucursales a través de Internet:** una compañía puede construir una red privada virtual sobre Internet o a través de una WAN pública. Esto permite a un negocio apoyarse firmemente en Internet y reducir su necesidad de una red privada, ahorrando costes y gestión de red suplementaria.
- **Acceso remoto seguro a través de Internet:** un usuario final cuyo sistema está equipado con protocolos de seguridad IP puede hacer llamadas locales a su proveedor de servicios Internet (PSI) y acceder de forma segura a una red de una compañía. Esto reduce el coste de los gastos de peaje de los empleados de viaje y de los abonados.
- **Establecimiento de conectividad intranet y extranet con asociados:** IPSec se puede utilizar para hacer las comunicaciones seguras con otras organizaciones, asegurando la autenticación y la privacidad y proporcionando un mecanismo de intercambio de claves.
- **Mejorando la seguridad en el comercio electrónico:** incluso aunque algunas aplicaciones Web y de comercio electrónico tienen protocolos de seguridad internos, la utilización de IPSec mejora esa seguridad.

La principal característica de IPSec que le permite soportar estas aplicaciones variadas es que puede cifrar y/o autenticar *todo* el tráfico a nivel IP. Así, todas las aplicaciones distribuidas, incluyendo la conexión remota, cliente/servidor, correo electrónico, transferencia de ficheros, acceso a la Web y así muchas más, se pueden hacer seguras cinco propuestas de normalizaciones relacionadas con seguridad y que definían las capacidades de seguridad en la capa internet. Los documentos producidos eran:

EL ÁMBITO DE IPSec

IPSec proporciona tres facilidades principales: una función de sólo autenticación conocida como Cabeza de Autenticación (AH, Authentication Header), una función combinada de autenticación/cifrado llamada Encapsulado de seguridad de la carga útil (ESP, Encapsulating Security Payload) y una función de intercambio de claves. Para redes privadas virtuales generalmente se desea autenticación y cifrado, ya que es importante (1) asegurar que usuarios no autorizados no entran en la red privada virtual y (2) asegura que si hay personas dedicadas a hacer escuchas en Internet no pueden leer los mensajes enviados por la red privada virtual. Ya que ambas características son deseables, la mayoría de las implementaciones utilizan ESP en lugar de AH. La función de intercambio de claves permite el intercambio manual de claves así como un esquema automático.

Las especificaciones IPsec son bastante complejas y se tratan en muchos documentos. Los más importantes de éstos, publicados en noviembre de 1998, son los RFCs 2401, 2402, 2406 y 2408 (véase Apéndice B). En esta sección se proporciona un repaso general a los elementos más importantes de IPsec.

ASOCIACIONES DE SEGURIDAD

Un concepto clave que aparece tanto en los mecanismos de autenticación como de privacidad en IP es la asociación de seguridad (SA, Security Association). Una asociación es una relación en un solo sentido entre un emisor y un receptor que proporciona servicios de seguridad al tráfico que se transporta. Si se necesita una relación paritaria, para un intercambio seguro en dos sentidos, entonces se requieren dos asociaciones de seguridad. Los servicios de seguridad se proporcionan a una SA para que utilice AH o ESP, pero no ambos.

Una asociación de seguridad está identificada unívocamente por tres parámetros:

- **Un índice de parámetros de seguridad (SPI, Security Parameters Index):** una cadena de bits asignada a esta SA y con significado local solamente. El SPI se transporta en las cabeceras AH y SPI para permitir que sistema receptor seleccione la SA bajo la que se procesarán los paquetes recibidos.
- **Dirección IP destino:** actualmente sólo se permiten direcciones monodistribución; ésta es la dirección del sistema final destino de la SA, que puede ser un usuario final o un sistema de red, como es un cortafuegos o un dispositivo de encaminamiento.
- **Identificador del protocolo de seguridad:** esto indica si la asociación es una asociación de seguridad AH o ESP.

Por lo tanto, en cualquier paquete IP, la asociación de seguridad está unívocamente identificada por la dirección destino en la cabecera IPv4 o IPv6 y el SPI incluida en la cabecera de extensión (AH o ESP).

Una implementación IPsec incluye una base de datos de asociaciones de seguridad que define los parámetros asociados con cada SA. Una asociación de seguridad se define normalmente por los parámetros siguientes:

- **Contador de número de secuencia:** un valor de 32 bits utilizado para generar el campo número de secuencia en las cabeceras AH o ESP.
- **Desbordamiento del contador de secuencia:** un indicador que avisa si se debe generar un evento audible si se produce un desbordamiento del contador de números de secuencia y así prevenir de transmisiones de paquetes adicionales de esta SA.
- **Ventana de anti-repeticiones:** utilizada para determinar si un paquete AH o ESP que llega es una repetición, mediante la definición de una ventana deslizante dentro de la cual se tiene que encontrar el número de secuencia.
- **Información AH:** algoritmo de autenticación, claves, tiempos de vida de las claves y parámetros relacionados que se utilizan con AH.
- **Información ESP:** algoritmos de cifrado y autenticación, claves, valores de inicialización, tiempos de vida de las claves y parámetros relacionados que se utilizan con ESP.
- **Tiempo de vida de la asociación de seguridad:** un intervalo de tiempo o contador de bytes después del cual una SA se tiene que reemplazar por una SA nueva (y un nuevo SPI) o terminar, más una indicación de cuál de estas opciones debería ocurrir.
- **Modo de protocolo IPsec:** túnel, transporte o marca de ambos (necesario en todas las implementaciones). Estos modos se discuten más tarde en esta sección.

- **MTU del camino:** cualquier unidad de transferencia máxima observada en el camino (tamaño máximo de los paquetes que se pueden transmitir sin fragmentación) y variables de caducidad (necesario en todas las implementaciones).

El mecanismo de gestión de claves que se utiliza para distribuir las claves está acoplado a los mecanismos de autenticación y de privacidad solamente a través del índice de parámetros de seguridad. Por lo tanto, la autenticación y la privacidad han sido especificadas independientemente de cualquier mecanismo de gestión de claves.

MODOS DE TRANSPORTE Y MODOS TÚNEL

AH y ESP permiten dos modos de uso: modo de transporte y modo túnel.

Modo transporte ESP

El modo transporte proporciona protección principalmente a los protocolos de las capas superiores. Es decir, la protección del modo de transporte se extiende sea la carga útil de un paquete IP. Algunos ejemplos incluyen a segmentos TCP o UDP o paquetes ICMP que operan directamente encima de IP en la pila de protocolos de un computador. Normalmente, el modo transporte se utiliza en comunicaciones extremo-a-extremo entre dos computadores (por ejemplo, un cliente y un servidor o dos estaciones de trabajo). Cuando ambos computadores implementan AH o ESP sobre IPv4 la carga útil son los datos que siguen a la cabecera IP. Para IPv6 la carga útil son los datos que siguen a la cabecera IP y a cualquier cabecera de extensión que esté presente, con la posible excepción de la cabecera de las opciones para el destino, que se podría incluir en la protección.

ESP en modo transporte cifra y opcionalmente autentifica la carga útil de IP pero no la cabecera IP. AH en modo transporte autentifica la carga útil de IP y porciones seleccionadas de la cabecera IP.

Modo túnel ESP

El modo túnel proporciona protección al paquete IP entero. Para alcanzar esto, después de que los campos AH o ESP se han incorporado al paquete IP, el paquete entero más un campo de seguridad se tratan como la carga útil de un paquete IP «exterior» nuevo con una cabecera IP exterior nueva. El paquete original entero, o interior, viaja a través del túnel desde un punto de la red IP a otro, ningún dispositivo de encaminamiento a lo largo del camino es capaz de examinar la cabecera IP interior. Ya que el paquete original está encapsulado el nuevo, un paquete más grande podría tener direcciones origen y destino totalmente diferentes, añadiendo seguridad. El modo túnel se utiliza cuando uno o ambos extremos de una SA es una pasarela de seguridad, como son un cortafuegos o un dispositivo de encaminamiento que implementa IPSec. Con el modo túnel, un determinado número de computadores en la red y detrás del cortafuegos pueden estar implicados en comunicaciones seguras sin implementar IPSec. Los paquetes no protegidos generados por tales computadores se transmiten mediante un túnel a través de redes externas mediante SA en modo túnel establecidas por el software IPSec en el cortafuegos o el dispositivo de encaminamiento seguro en las fronteras de la red local.

A continuación se da un ejemplo de cómo opera IPSec en modo túnel. Un computador A en una red genera un paquete IP con dirección destino del computador B en otra red. El paquete se encamina desde el computador origen a un cortafuegos o un dispositivo de encaminamiento seguro en la frontera de la red de A. El cortafuegos filtra todos los paquetes de salida para determinar si necesitan procesamiento IPSec. Si este paquete de A a B requiere IPSec, el cortafuegos realiza el procesamiento IPSec y encapsula el paquete en un paquete IP exterior. La dirección IP origen de este paquete exterior es la del cortafuegos y la dirección destino podría ser la de un cortafuegos que constituye la frontera de la red local de B. Este paquete se encamina al cortafuegos de B, y los dispositivos de encaminamiento intermedios sólo examinan la cabecera IP exterior. En el cortafuegos de B, se elimina la cabecera exterior y se encamina el paquete interior a B.

ESP en modo túnel cifra y opcionalmente autentifica al paquete IP interior completo, incluyendo la cabecera IP interior. AH en modo túnel autentifica el paquete IP interior completo y porciones seleccionadas de la cabecera IP exterior.

CABECERA DE AUTENTIFICACIÓN

La cabecera de autenticación proporciona un medio para la integridad de los datos y la autenticación de los paquetes IP. La característica de integridad de los datos asegura que la modificación no detectada del contenido del paquete no es posible en su camino. La característica de autenticación habilita a un sistema final o a un dispositivo de red autenticar el usuario o la aplicación y filtrar el tráfico adecuadamente; también previene el ataque de suplantación de dirección observado actualmente en Internet. La AH también protege frente a ataques de repetición descritos más tarde en esta sección.

La autenticación se basa en el uso de un código de autenticación de mensaje (MAC, Message Authentication Code), como se describe en la Sección 18.3; por lo tanto las dos partes deben compartir una clave secreta.

La cabecera de autenticación consta de los siguientes campos (Figura 18.14):

- **Cabecera siguiente (8 bits):** identifica el tipo de cabecera que viene a continuación de ésta.
- **Longitud de la carga útil (8 bits):** longitud de la cabecera de autenticación en palabras de 32 bits, menos 2. Por ejemplo, la longitud por defecto del campo de datos de autenticación es de 96 bits, o tres palabras de 32 bits. Con una cabecera de 3 palabras fijas, hay un total de seis palabras en la cabecera, por tanto el campo longitud de la carga útil vale 4.
- **Reservado (16 bits):** para usos futuros.
- **Índice de parámetros de seguridad (32 bits):** identifica a una asociación de seguridad.
- **Número de secuencia (32 bits):** el valor de un contador que se incrementa monotónicamente.
- **Datos de autenticación (variable):** un campo de longitud variable (debe ser número entero de palabras de 32 bits) que contiene el valor de chequeo de integridad (ICV, Integrity Check Values), o el MAC para este paquete.

El contenido del campo de datos de autenticación se calcula sobre lo siguiente:

- Campos de la cabecera IP que no cambian en el camino (inmutables) o que tienen un valor predecible de AH SA cuando llega al sistema final. Los campos que pueden cambiar en el tránsito o que no se pueden predecir en la llegada se hacen cero para motivos de cálculo tanto en el origen como en la fuente.

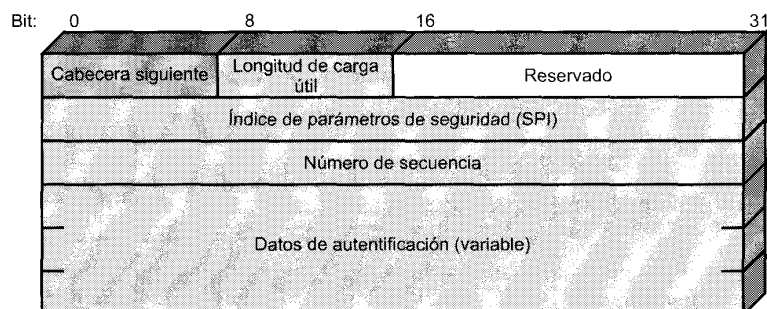


Figura 18.14. Cabecera de autenticación IPSec.

- La cabecera AH excluyendo al campo datos de autenticación. El campo de datos de autenticación se hace cero para motivos de cálculo tanto en el origen como en la fuente.
- Los datos del protocolo de la capa superior, que se supone son inmutables en el camino (por ejemplo, un segmento TCP o un paquete IP interior en modo túnel).

Para IPv4, algunos ejemplos de campos inmutables son la longitud de la cabecera IP y la dirección origen. Como ejemplo de campo sujeto a cambios pero predecible es la dirección destino (para el enca minamiento por la fuente estricto o aproximado). Ejemplos de campos sujetos a cambios que se hacen cero antes de los cálculos ICV son los campos tiempo de vida y suma de comprobación. Nótese que los campos de dirección origen y destino están protegidos, y así se previene la suplantación de dirección.

Para IPv6, algunos ejemplos de la cabecera base son *versión* (inmutable), *dirección destino* (sujeto a cambios pero predecible) *etiqueta de flujo* (sujeto a cambios y puesto a cero para los cálculos).

ENCAPSULADO DE SEGURIDAD DE LA CARGA ÚTIL

El encapsulado de seguridad de la carga útil proporciona servicios de privacidad, incluyendo confidencialidad de los contenidos de los mensajes y una confidencialidad limitada del flujo de tráfico. Como una característica opcional, ESP puede también proporcionar los mismos servicios de autenticación que AH.

La Figura 18.15 muestra el formato de un paquete. Contiene los siguientes campos:

- **Índice de parámetros de seguridad (32 bits):** identifica una asociación de seguridad.
- **Número de secuencia (32 bits):** el valor de un contador que se incrementa monóticamente.
- **Datos de carga útil (variable):** esto es un segmento de la capa de transporte (modo de transporte) o un paquete IP (modo túnel) que se protege mediante el cifrado.
- **Relleno (0-255 bytes):** este campo se puede requerir si el algoritmo de cifrado requiere que le texto nativo sea un múltiplo de algún número de octetos.
- **Longitud de relleno (8 bits):** indica el número de bytes de relleno en el campo que precede a éste.
- **Cabecera siguiente (8 bits):** identifica el tipo de datos contenidos en el campo de datos de carga útil mediante la identificación de la primera cabecera en esa carga útil (por ejemplo, una cabecera de extensión en IPv6 o un protocolo de la capa superior como TCP).

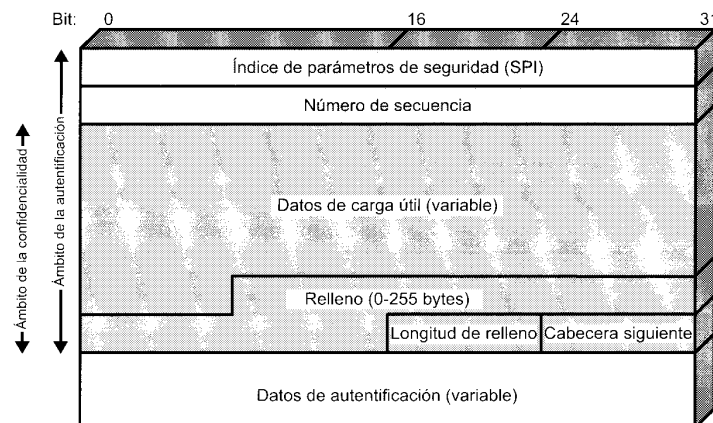


Figura 18.15. Formato ESP IPSec.

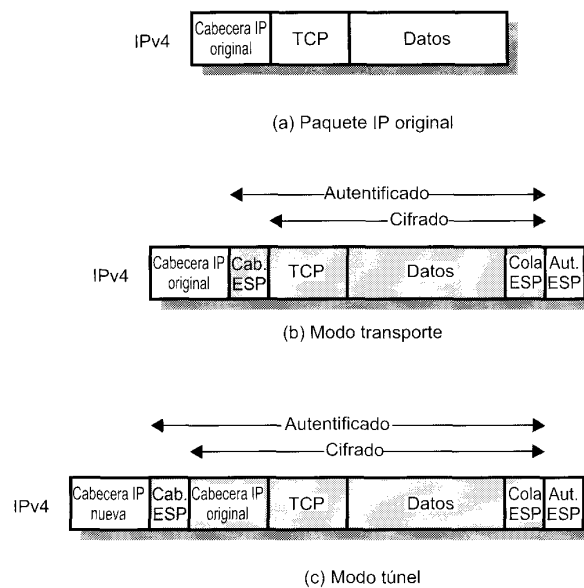


Figura 18.16. Ámbito del cifrado y la autenticación ESP.

- **Datos de autenticación (variable):** un campo de longitud variable (debe ser número entero de palabras de 32 bits) que contiene el valor de chequeo de integridad calculado sobre el paquete ESP menos el campo de datos de autenticación.

La Figura 18.16 muestra el ámbito del cifrado y autenticación ESP en los modos de transporte y túnel.

GESTIÓN DE CLAVES

La porción de gestión de claves de IPsec supone la determinación y distribución de claves secretas. Los documentos de la arquitectura de IPsec obligan a permitir dos tipos de gestión de claves:

- **Manual:** un administrador del sistema configura manualmente cada sistema con sus propias claves y con las claves de otros sistemas de comunicación. Esto es práctico para entornos pequeños y relativamente estáticos.
- **Automática:** un sistema automático habilita la creación bajo demanda de claves para SAs y facilita el uso de claves en sistemas distribuidos grandes con una configuración cambiante. Un sistema automático es lo más flexible pero requiere más esfuerzos para configurar y requiere más software, por lo tanto es más probable que las instalaciones más pequeñas opten por una gestión de claves manual.

El protocolo de gestión de claves automático que se utiliza por defecto en IPsec se conoce como ISAKMP/Oakley, que consta de los siguientes elementos:

- **Protocolo de determinación de claves Oakley:** Oakley es un protocolo de intercambio de claves basado en el algoritmo Diffie-Hellman, pero proporcionando una seguridad adicional. En particular, Diffie-Hellman sólo no autentifica a los dos usuarios que intercambian claves, haciendo el protocolo vulnerable a la suplantación. Oakley incluye mecanismos para autenticar a los usuarios.

- **Asociación de Seguridad de Internet y protocolo de Gestión de claves (ISAKMP, Internet Security Association and Key Management Protocol):** ISAKMP proporciona un entorno de trabajo para la gestión de claves en Internet y proporciona el soporte del protocolo específico, incluyendo formatos, para la negociación de los atributos de seguridad.

ISAKMP por sí mismo no impone un algoritmo de intercambio de claves específico; en lugar de eso, ISAKMP consiste de un conjunto de tipos de mensajes que permiten el uso de una variedad de algoritmos de intercambio de claves. Oakley es el algoritmo de intercambio de claves de uso obligatorio en la versión inicial de ISAKMP.

18.6. LECTURAS RECOMENDADAS Y PÁGINAS WEB

Los tópicos de este capítulo se tratan con un mayor detalle en [STAL95]. Para una mayor información sobre los algoritmos de criptografía, [SCHN96] es un trabajo de referencia esencial; contiene descripciones de virtualmente cada algoritmo y protocolo criptográfico publicado en los últimos 15 años.

SCHN96 Schneier, B. *Applied Cryptography*. New York: Wiley, 1996.

STAL99 Stallings, W. *Cryptography and Network Security: Principles and Practice, 2nd Edition*. Upper Saddle River, NJ: Prentice Hall, 1999.



SITIOS WEB RECOMENDADOS

- **COAST:** un conjunto completo de enlaces relacionados con la criptografía y la seguridad de red.
- **Área de seguridad de la IEFT:** proporciona información actualizada sobre los esfuerzos en la estandarización de la seguridad en Internet.
- **Comité Técnico de IEEE sobre seguridad y privacidad:** proporciona copias de periódicos de noticias de IEEE e información sobre las actividades relacionadas de IEEE.

18.7. PROBLEMAS

- 18.1. Dé algunos ejemplos donde el análisis del tráfico pueda comprometer la seguridad. Describa situaciones donde el cifrado extremo-a-extremo combinado con el cifrado de enlace permitiría todavía bastante análisis del tráfico como para que sea peligrosos.
- 18.2. Los esquemas de distribución de claves utilizando un centro de control de accesos y/o un centro de distribución de claves tienen puntos centrales vulnerables a ataques. Discuta las implicaciones en la seguridad de tal centralización.
- 18.3. Suponga que alguien sugiere la siguiente forma para confirmar que usted y su pareja están en posesión de la misma clave secreta. Usted crea una cadena de bits aleatoria de longitud igual a la de la clave, realiza la operación XOR entre la cadena y la clave y envía el resultado por el canal. Su pareja realiza la operación XOR del bloque recibido con la clave (que debería ser la misma) y envía el resultado de vuelta. Usted comprueba y si recibe la cadena de bits original, se ha verificado que su pareja tiene la misma clave sin haberla transmitido en ningún momento. ¿Hay algún defecto en este esquema?

- 18.4.** Antes del descubrimiento de cualquier esquema de clave pública específico, como es RSA, se desarrolló una demostración de existencia cuyo propósito fue demostrar que el cifrado de clave pública era posible en teoría. Considere la función $f_1(x_1) = z_1$; $f_2(x_2, y_2) = z_2$; $f_3(x_3, y_3) = z_3$ cuyos valores son enteros con $1 \leq x_i, y_i, z_i \leq N$. La función f_1 se puede representar por un vector $\mathbf{M}_1$ de longitud N , en el que la entrada k es el valor de $f_1(k)$. De igual forma, f_2 y f_3 se pueden representar por las matrices $N \times N$ $\mathbf{M}_2$ y $\mathbf{M}_3$. La intención es representar el proceso de cifrado/descifrado por búsquedas en tablas con valores muy grandes de N . Tales tablas serían imposibles de grandes pero podrían, en principio, construirse. El esquema trabaja como sigue: construya $\mathbf{M}_1$ con permutaciones aleatoria de todos los enteros entre 1 y N ; esto es, cada entero aparece exactamente una sola vez en $\mathbf{M}_1$. Construya $\mathbf{M}_2$ de forma que cada fila contenga una permutación aleatoria de los N primeros enteros. Finalmente, rellene $\mathbf{M}_3$ para que satisfaga la siguiente condición:

$$f_3(f_2(f_1(k), p), k) = p \text{ para todo } k, p \text{ con } 1 \leq k, p \leq N$$

En otras palabras:

1. $\mathbf{M}_1$ toma como entrada k y produce una salida x .
2. $\mathbf{M}_2$ toma como entrada x y p dando z como salida
3. $\mathbf{M}_3$ toma como entrada z y k y produce p .

Las tres tablas, una vez construidas, se hacen públicas.

- a) Debería quedar claro que es imposible construir $\mathbf{M}_3$ para que satisfaga la condición anterior. Como ejemplo, rellene $\mathbf{M}_3$ para el caso simple siguiente:

$M_1 =$	5	$M_2 =$	5	2	3	4	1	$M_3 =$					
	4		4	2	5	1	3						
	2		1	3	2	4	5						
	3		3	1	4	2	5						
	1		2	5	3	4	1						

Convención: el elemento i de $\mathbf{M}_1$ corresponde a $k = 1$. La fila i de $\mathbf{M}_2$ corresponde a $x = i$; la columna j de $\mathbf{M}_2$ corresponde a $p = j$. La fila i de $\mathbf{M}_3$ corresponde a $z = i$; la columna j de $\mathbf{M}_3$ corresponde a $k = j$.

- b) Describa la utilización de este conjunto de tablas para llevar a cabo el cifrado y el descifrado entre dos usuarios.
- c) Argumente que esto es un esquema seguro.
- 18.5.** Lleve a cabo el cifrado y el descifrado utilizando el algoritmo RSA como en la Figura 18.12, con los siguientes valores:
- a) $p = 3$; $q = 11$, $d = 7$; $M = 5$
 - b) $p = 5$; $q = 11$, $d = 3$; $M = 9$
 - c) $p = 7$; $q = 11$, $d = 17$; $M = 8$
 - d) $p = 11$; $q = 13$, $d = 11$; $M = 7$
 - e) $p = 17$; $q = 31$, $d = 7$; $M = 2$. *Sugerencia:* el descifrado no es tan difícil como se piensa; utilice alguna sutileza.
- 18.6.** En un sistema de clave pública utilizando RSA, intercepta el texto cifrado $C = 10$ enviado a un usuario cuya clave pública es $e = 5$, $n = 35$. ¿Cuál es el texto nativo M ?

- 18.7.** En un sistema RSA, la clave pública de un usuario dado es $e = 31$, $n = 3.599$. ¿Cuál es la clave privada de este usuario?
- 18.8.** Suponga que se tiene un conjunto de bloques codificado con el algoritmo RSA y que no se tiene la clave privada. Suponga que $n = pq$, e es la clave pública. Suponga también que alguien comenta que conoce que uno de los bloques de texto nativo tiene un factor común con n . ¿Puede esto ayudar de alguna forma?
- 18.9.** Muestre como RSA se puede representar por las matrices **M1**, **M2** y **M3** del Problema 18.4.
- 18.10.** Considere el siguiente esquema:
1. Elija un número impar, E .
 2. Elija dos números primos, P y Q , donde $(P - 1)(Q - 1) - 1$ es divisible por E de modo uniforme.
 3. Multiplique P y Q para obtener N .
 4. Calcule $D = \frac{(P - 1)(Q - 1)(E - 1) + 1}{E}$.
- ¿Es equivalente este esquema a RSA? Muestre por qué sí o por qué no.
- 18.11.** Considere la utilización de RSA con una clave conocida para construir una función de dispersión de un solo sentido. Después, procese un mensaje constituido por una secuencia de bloques como sigue: cifre el primer bloque, realice la operación XOR entre el resultado y el segundo bloque y cifrelo de nuevo, etc. Muestre que este esquema no es seguro mediante la resolución del siguiente problema. Dado un mensaje de dos bloques, $B1$ y $B2$, y su función de dispersión:

$$\text{RSAH}(B1, B2) = \text{RSA}(\text{RSA}(B1) \oplus B2)$$

Dado un bloque arbitrario $C1$, elija $C2$ para que $\text{RSAH}(C1, C2) = \text{RSAH}(B1, B2)$.

Aplicaciones distribuidas

19.1. Notación Sintáctica Abstracta Uno (ASN.1)

Sintaxis abstracta
Conceptos de ASN.1

19.2. Gestión de red—SNMP

Sistemas de gestión de red
Protocolo simple de gestión de red versión 2 (SNMPv2)
Protocolo simple de gestión de red versión 3 (SNMPv3)

19.3. Correo electrónico—SMTP y MIME

Protocolo sencillo de transferencia de correo (SMTP)
Ampliación de correo Internet multiobjetivo (MIME)

19.4. Protocolo de transferencia de hipertextos (HTTP)

Descripción general de HTTP
Mensajes
Mensajes de petición
Mensajes de respuesta
Entidades

19.5. Lecturas recomendadas y páginas Web

19.6. Problemas

- [illegible]

[illegible]

19.1. NOTACIÓN SINTÁCTICA ABSTRACTA UNO (ASN.1)

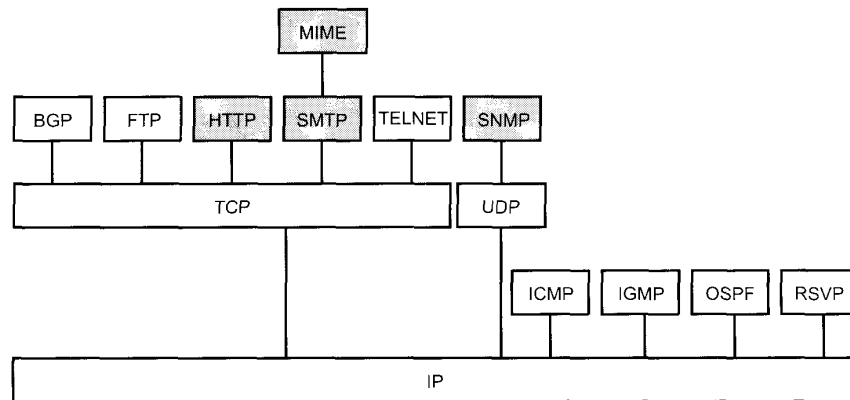


Figura 19.1. Protocolos del nivel de aplicación en su contexto.

SINTAXIS ABSTRACTA

La Tabla 19.1 define algunos términos claves que son relevantes en una discusión de ASN.1 y la Figura 19.2 muestra los conceptos subyacentes.

Para los objetivos de esta discusión, se puede considerar que una arquitectura de comunicaciones en un sistema final tiene dos componentes principales. La componente transferencia de datos está relacionada con los mecanismos de transferencia de datos entre sistemas finales. En el caso de la familia de protocolos TCP/IP, este componente está integrado por los elementos de TCP o UDP y los protocolos de capas más bajas. En el caso de la arquitectura OSI, estaría formado por la capa de sesión hacia abajo. El componente de aplicación es el usuario del componente de transferencia de datos y está relacionado con la aplicación del usuario final. En el caso del conjunto de protocolos TCP/IP, este componente es una aplicación tal como SNMP, FTP, SMTP o TELNET. En el caso de la arquitectura OSI, el componente consiste en la capa de aplicación, que está compuesta de una serie de elementos de servicio de aplicaciones, y la capa de presentación.

Tabla 19.1. Términos relevantes de ASN.1.

Sintaxis abstracta	Describe la estructura genérica de los datos independientemente de cualquier técnica de codificación utilizada para representar los datos. La sintaxis permite que se definan tipos de datos y valores de aquellos tipos que se van a especificar.
Tipo de datos	Un conjunto con nombre de valores. Un tipo puede ser simple, que se define especificando el conjunto de sus valores, o estructurado, que se define en términos de otros tipos.
Codificación	La secuencia completa de octetos utilizados para representar un valor de un dato.
Reglas de codificación	Una especificación de la traducción de una sintaxis a otra. Concretando, las reglas de codificación determinan algorítmicamente, para cualquier conjunto de valores de datos definidos en la sintaxis abstracta, la representación de aquellos valores en una sintaxis de transferencia.
Sintaxis de transferencia	Las formas en las que los datos se representan realmente en términos de patrones de bits mientras están en tránsito entre entidades de presentación.

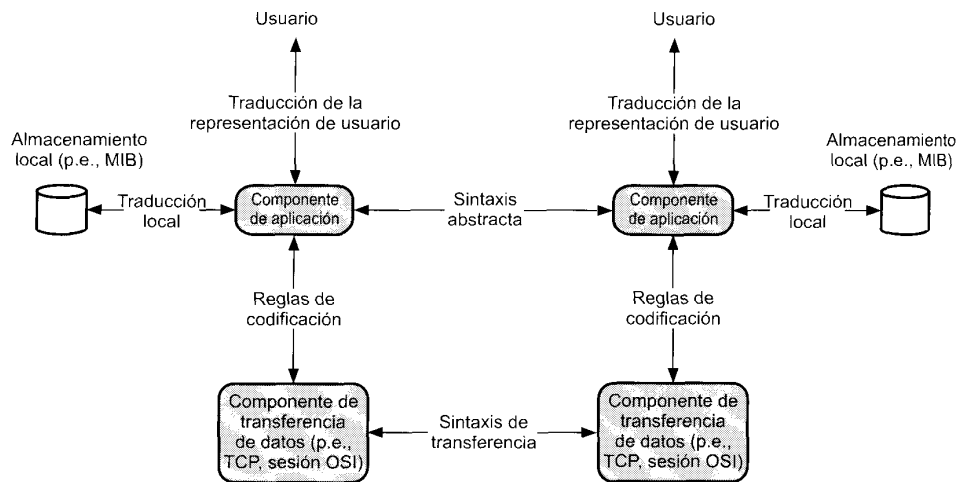


Figura 19.2. El uso de la sintaxis abstracta y de transferencia.

Conforme cruzamos el límite entre el componente de aplicación y el de transferencia, existe un cambio significativo en la forma en que se ven los datos. Para el componente de transferencia de datos, los datos recibidos de una aplicación se especifican como el valor binario de una secuencia de octetos. Este valor binario se puede ensamblar directamente en unidades de datos de servicio (SDU) para transferirlos entre capas y en unidades de datos de protocolo (PDU) para transferirlos entre entidades de protocolo dentro de una capa. El componente de aplicación, sin embargo, está relacionado con la visión de los datos que tiene un usuario. En general, este punto de vista corresponde a un conjunto estructurado de información, como un texto en un documento, un fichero personal, una base de datos integrada, o una presentación visual de información en imágenes. El usuario está relacionado principalmente con la semántica de los datos. El componente de aplicación debe proporcionar una representación de estos datos que se puedan convertir en valores binarios; esto es, debe estar relacionado con la sintaxis de los datos.

El enfoque mostrado en la Figura 19.2 para dar soporte a los datos de aplicación es como sigue. Para el componente de aplicación, la información se representa en una sintaxis abstracta que trata con tipos de datos y valores de los datos. La sintaxis abstracta especifica formalmente los datos independientemente de cualquier representación específica. Así, una sintaxis abstracta tiene muchas similitudes con los aspectos de definición de los tipos de datos de los lenguajes de programación convencionales, como Pascal, C y Ada, y de las gramáticas como la descrita con la Forma Backus-Naur (BNF). Los protocolos de aplicación describen sus PDU en términos de una sintaxis abstracta.

Esta sintaxis abstracta se utiliza para el intercambio de información entre componentes de aplicaciones en sistemas diferentes. El intercambio se efectúa con PDU del nivel de aplicación, que contienen información de control del protocolo y datos de usuario. Dentro de un sistema, la información representada utilizando sintaxis abstracta se debe traducir en alguna forma de representación para el usuario humano. De igual forma, esta sintaxis abstracta se debe traducir en algún formato local para su almacenamiento. Por ejemplo, este tipo de traducción se utiliza en el caso de la información de gestión de red. Además, está llegando a ser común el uso de la sintaxis abstracta para definir los elementos de los datos en el almacenamiento local. Así, la notación sintáctica abstracta la utilizan los usuarios para definir la información de gestión de red; la aplicación debe entonces convertir esta definición en una forma conveniente para el almacenamiento local.

El componente debe también traducir entre la sintaxis abstracta de la aplicación y una sintaxis de transferencia que describa los valores de los datos en una forma binaria, adecuada para la interacción

con el componente de transferencia de datos. Por ejemplo, una sintaxis abstracta puede incluir un tipo de dato carácter; la sintaxis de transferencia podría especificar IRA o EBCDIC para la codificación.

La sintaxis de transferencia define así la representación de los datos que se van a intercambiar entre componentes de transferencia de datos. La traducción de la sintaxis abstracta a la sintaxis de transferencia se realiza por medio de reglas que especifican la representación de cada valor de los datos de cada tipo de datos.

Este enfoque para el intercambio de datos de aplicación resuelve dos problemas relacionados con la representación de datos en entornos distribuidos y heterogéneos.

- Existe una representación común para el intercambio de datos entre sistemas diferentes.
- Con respecto a la parte interna de un sistema, una aplicación utiliza alguna representación particular de los datos. El esquema de sintaxis abstracta de transferencia resuelve automáticamente la diferencias entre las representaciones de entidades de aplicación cooperantes.

El requisito fundamental para la selección de una sintaxis de transferencia es que permita la correspondiente sintaxis abstracta. Además, la sintaxis de transferencia puede tener otros atributos que no están relacionados con las sintaxis abstractas que puede soportar. Por ejemplo, una sintaxis abstracta se puede soportar por una de las cuatro sintaxis de transferencia, que son iguales en todos los aspectos excepto en que una proporciona compresión de datos, otra encriptado, otra ambas y la última ninguna. La elección de la sintaxis de transferencia que se va a utilizar depende de consideraciones de coste y seguridad.

CONCEPTOS DE ASN.1

El bloque básico de construcción de una especificación ASN.1 es el módulo. Comenzamos esta sección examinando el nivel más alto de la estructura del módulo. Después, se introducen algunas convenciones lexicográficas utilizadas en las definiciones ASN.1. A continuación se describen los tipos de datos definidos en ASN.1. Finalmente, se dan ejemplos del uso de ASN.1.

Definición de módulos

ASN.1 es un lenguaje que puede ser utilizado para definir estructuras de datos. La definición de una estructura se hace asignando un nombre al módulo. El nombre del módulo se puede utilizar después para referenciar la estructura. Por ejemplo, el nombre del módulo se puede utilizar como un nombre sintáctico abstracto; una aplicación puede pasar este nombre al servicio de presentación para especificar la sintaxis abstracta de las PDU de la aplicación que ésta desea intercambiar con una entidad de aplicación paritaria.

Los módulos tienen la siguiente forma:

```
<referencia-de-módulo> DEFINITIONS ::=
BEGIN
  EXPORTS
  IMPORTS
  Lista de asignación
End
```

El rótulo «referencia-de-módulo» es un nombre de módulo seguido opcionalmente por un identificador de objeto para identificar el módulo. La construcción EXPORTS indica que definiciones de este módulo se pueden importar por otros módulos. La construcción IMPORTS indica definiciones de tipos y de valores de otros módulos que se van a importar en este módulo. Ni la construcción IMPORTS ni la EXPORTS pueden ser incluidas a menos que el identificador de objeto del módulo esté incluido. Finalmente, la «Lista-de-asignación» contiene asignaciones de tipo, asignaciones de valores y definiciones de

macro. Las definiciones de macro se discuten después en esta sección. Las asignaciones de tipo y de valores tienen la forma:

⟨nombre⟩ :: = ⟨descripción⟩

La forma más fácil de describir la sintaxis es con un ejemplo. Primero, necesitamos especificar algunas convenciones léxicas.

Convenciones léxicas

Las estructuras, los tipos y los valores de ASN.1 se expresan en una notación similar a la de un lenguaje de programación. Las convenciones léxicas que se siguen son:

1. La disposición no es importante; los espacios múltiples y las líneas en blanco se consideran como un único espacio.
2. Los comentarios están delimitados por un par de guiones (--) al comienzo y al final del comentario o por un par de guiones al comienzo del comentario y el final de la línea actúa como fin del comentario.
3. Los identificadores (nombres de valores y campos), las referencias de tipo (nombres de tipos) y los nombres de módulo constan de letras mayúsculas, minúsculas, dígitos y guiones.
4. Un identificador comienza por una letra minúscula.
5. Una referencia de tipo o un nombre de módulo comienza con un letra mayúscula.
6. Un tipo interior consta de letras, todas mayúsculas. Un tipo interior es un tipo comúnmente utilizado para el que se proporciona una notación normalizada.

Tipos de datos abstractos

ASN.1 es una notación para tipos de datos abstractos y sus valores. Un tipo se puede ver como una colección de valores. El número de valores que puede tomar un tipo puede ser infinito. Por ejemplo, el tipo INTEGER tiene un número infinito de valores.

Podemos clasificar los tipos en cuatro categorías:

- **Sencillo:** son tipos atómicos sin componentes.
- **Estructurado:** un tipo estructurado tienen componentes.
- **Marcado («tagged»):** estos tipos son derivados de otros tipos.
- **Otro:** esta categoría incluye los tipos CHOICE y ANY, discutidos posteriormente en esta sección.

Cada tipo de datos ASN.1, con excepción de CHOICE y ANY, tiene asociado una marca. La marca consta de un nombre de clase y un número de marca entero no negativo. Existen cuatro clases de tipos de datos, o cuatro clases de marcas:

- **Universal:** tipos independientes de la aplicación y de los mecanismos de construcción generalmente útiles en cualquier contexto; éstos se incluyen en el estándar y se muestran en la Tabla 19.2.
- **Aplicación amplia:** relevantes en aplicaciones particulares; éstos están definidos en otros estándares.
- **Específico del contexto:** también relevantes para aplicaciones particulares pero aplicables en un contexto limitado.
- **Privado:** tipos definidos por usuarios que no se encuentran en ningún estándar.

Un tipo de datos está identificado únicamente por su marca. Los tipos de ASN.1 son los mismos si y sólo si sus números de marca son los mismos. Por ejemplo, UNIVERSAL 4 se refiere a OctetString, que es de la clase UNIVERSAL y tiene número de marca 4 dentro de la clase.

Tabla 19.2. Asignación de etiquetas de clase universal

Etiqueta	Nombre de tipo	Conjunto de valores
Tipos básicos		
UNIVERSAL 1 UNIVERSAL 2	Boolean Integer	VERDADERO o FALSO El conjunto completo de número positivos y negativo, incluyendo el cero.
UNIVERSAL 3 UNIVERSAL 4 UNIVERSAL 9 UNIVERSAL 10	Bit String Octet String Real Enumerated	Una secuencia de cero o más bits. Una secuencia de cero o más octetos. Números reales. Una lista explícita de valores de enteros que pueden tomar una representación de tipo de datos.
Tipos de objetos		
UNIVERSAL 6 UNIVERSAL 7	Object Identifier Object Descriptor	El conjunto de valores asociados con objetos de información. Cada valor es texto legible por una persona proporcionando una breve descripción de un objeto de información.
Tipos de cadenas de caracteres		
UNIVERSAL 18 UNIVERSAL 19 UNIVERSAL 20	NumericString PrintableString TeletexString	Dígitos de 0 a 9, espacio. Caracteres imprimibles. Conjunto de caracteres definidos en la Recomendación T.61 de la CCITT.
UNIVERSAL 21	VideotexString	Conjunto de caracteres del alfabeto y gráficos definidos en Recomendación T.100 y T.101 de la CCITT.
UNIVERSAL 22 UNIVERSAL 25 UNIVERSAL 26 UNIVERSAL 27	IA5String GraphicString VisibleString GeneralString	Alfabeto Internacional Cinco (equivalente a ASCII). Conjunto de Caracteres definidos en ISO 8824. Conjunto de Caracteres definidos en ISO 646 (equivalente a ASCII). Cadena de caracteres general.
Tipos misceláneos		
UNIVERSAL 5	NULL	El valor NULL. Utilizado comúnmente donde son posibles varias alternativas pero ninguna de ellas se aplica.
UNIVERSAL 8	EXTERNAL	Un tipo definido en algún documento externo. No necesita ser uno de los tipos ASN.1 válidos.
UNIVERSAL 23	UTCTime	Consta de la fecha, especificada con un año de dos dígitos, un mes de dos dígitos y un día de dos dígitos, seguido por la hora, especificada por las horas, minutos y opcionalmente por los segundos, seguido por una especificación opcional de la hora local diferente de la hora universal.
UNIVERSAL 24	Generalized Time	Consta de la fecha, especificada con un año de cuatro dígitos, un mes de dos dígitos y un día de dos dígitos, seguido por la hora, especificada por las horas, minutos y opcionalmente por los segundos, seguido por una especificación opcional de la hora local diferente de la hora universal.
UNIVERSAL 11-15 UNIVERSAL 28-	Reserved Reserved	Reservado para adendas al estándar ASN.1. Reservado para adendas al estándar ASN.1.
Tipos de estructuras		
UNIVERSAL 16	SEQUENCE y SEQUENCE OF	SECUENCIA: definida haciendo referencia a una lista ordenada y fija de tipos; cada valor es una lista ordenada de valores, uno de cada tipo de componente.
UNIVERSAL 17	SET y SET OF	SECUENCIA DE: definida haciendo referencia a un único tipo existente; cada valor es una lista ordenada de cero o más valores del tipo existente. CONJUNTO: definido haciendo referencia a una lista no ordenada y fija de tipos; algunos de los cuales se pueden declarar opcionales; cada valor es una lista no ordenada de valores, uno de cada tipo de componente. CONJUNTO DE: definido haciendo referencia a un único tipo existente; cada valor es una lista no ordenada de cero o más valores del tipo existente.

Un **tipo sencillo** es uno definido especificando directamente el conjunto de sus valores. Se pueden considerar estos tipos como tipos atómicos; todos los otros tipos están constituidos de tipos sencillos. Los tipos de datos sencillos en la clase UNIVERSAL se pueden agrupar en varias categorías, como se indica en la Tabla 19.2; éstas no son categorías «establecidas» en el estándar pero se utilizan aquí por conveniencia.

Al primer grupo de los tipos sencillos, a falta de un término mejor, se le puede denominar tipo básico. El tipo Boolean está claro. El tipo Integer es el conjunto de los enteros positivos y negativos y el cero. Además, se pueden asignar nombres a valores individuales de enteros para indicar un significado especial. El tipo BitString es un conjunto ordenado de cero o más bits; se pueden asignar nombres individuales a los bits. El valor real de un BitString se puede especificar como una cadena de dígitos binarios o hexadecimales. De igual forma, un tipo OctetString se puede especificar como una cadena de dígitos binarios o hexadecimales. El tipo de datos Real consta de números expresados en notación científica (mantisa, base, exponente); esto es:

$$M \times B^E$$

La mantisa (M) y el exponente (E) pueden tomar cualquier valor entero, positivo o negativo; se suele utilizar como base (B) los valores 2 o 10.

Finalmente, el tipo Enumerado consiste en una lista explícitamente enumerada de enteros, junto con un nombre asociado a cada entero. La misma funcionalidad se puede obtener con el tipo Integer dando nombre a algunos de los valores enteros; pero, debido a la utilidad de esta característica, se ha definido un tipo separado. Obsérvese, sin embargo, que aunque los valores de un tipo enumerado son enteros, no tienen una semántica entera. Esto es, no se pueden realizar operaciones aritméticas con los valores enumerados.

Los tipos de objetos se utilizan para nombrar y describir los objetos de información. Algunos ejemplos de objetos de información son documentos normalizados, sintaxis abstractas o de transferencia, estructuras de datos y objetos gestionados. En general, un objeto de información es una clase de información (por ejemplo, un formato de fichero) en lugar de un ejemplo de tal clase (por ejemplo, un fichero individual). El identificador de Objeto es un identificador único para un objeto particular. Su valor consiste en una secuencia de enteros. El conjunto de objetos definidos tiene una estructura en árbol, siendo la raíz de ese árbol el objeto referente al estándar ASN.1. Comenzando con la raíz del árbol identificador de objetos, cada valor de componente del identificador de objeto identifica un arco en el árbol. El descriptor Object es una descripción legible por un humano de un objeto de información.

ASN.1 define una serie de tipos de cadenas de caracteres. El valor de cada uno de estos tipos consta de una secuencia de cero o más caracteres elegidos de entre conjunto de caracteres normalizados.

Existen algunos tipos misceláneos que también se han definido en la clase UNIVERSAL. El tipo Null se utiliza para posiciones de las estructuras donde un valor puede o no puede estar presente. El tipo Null es simplemente la alternativa de que no esté presente un valor en esa posición en la estructura. Un tipo External es uno cuyos valores no están especificados en el estándar ASN.1; está definido en otro documento o estándar y se pueden definir utilizando una notación bien definida. UTCTime y GeneralizedTime son dos formatos diferentes para expresar el tiempo. En ambos casos se puede especificar un tiempo local o universal.

Los **tipos estructurados** son aquellos que consisten de componentes. ASN.1 proporciona cuatro tipos estructurados para construir tipos de datos complejos a partir de tipos de datos sencillos:

- SEQUENCE
- SEQUENCE OF
- SET
- SET OF

Los tipos SEQUENCE («secuencia») y SEQUENCE OF («secuencia de») se utilizan para definir una lista ordenada de valores de uno o más tipos de datos. Esto es análogo a la estructura registro de

muchos lenguajes de programación, como COBOL. Un tipo Sequence consta de una lista ordenada de elementos, cada uno especificando un tipo y, opcionalmente, un nombre. La notación para definir el tipo Sequence es como sigue:

```
SequenceType ::= SEQUENCE {ElementTypeList} | SEQUENCE { }
ElementTypeList ::= ElementType | ElementTypeList, ElementType
ElementType ::=
    NamedType                               |
    NamedType OPTIONAL                       |
    NamedType DEFAULT Value                 |
    COMPONENTS OF Type
```

donde la barra vertical indica una alternativa.

Un NamedType («tipodenominado») es una referencia de tipo con o sin nombre. Cada definición de elemento puede ser seguida por la palabra clave OPTIONAL («opcional») o DEFAULT («implícito»). La clave OPTIONAL indica que el elemento componente no necesita estar presente en un valor de secuencia. La clave DEFAULT indica que, si el elemento componente no está presente, entonces será asignado el valor especificado por la cláusula DEFAULT. La cláusula COMPONENT OF («compuesto de») se utiliza para definir la inclusión, en ese punto de la ElementTypeList («lista de tipos de elementos»), de todas las secuencias de ElementType que aparecen en el tipo referenciado.

Un tipo SEQUENCE OF consta de un número variable de elementos ordenados, todos del mismo tipo. Una definición de SEQUENCE OF tiene la siguiente forma:

```
SequenceOfType ::= SEQUENCE OF Type | SEQUENCE
```

La notación SEQUENCE se debe interpretar como SEQUENCE OF ANY («secuencia cualquiera»); el tipo ANY («cualquiera») se explicará en una subsección posterior.

Un tipo Set («conjunto») es similar a Sequence, excepto que el orden de los elementos no es significativo; los elementos pueden estar agrupados en cualquier orden cuando se codifican en una representación específica. Una definición de Set tiene la siguiente forma:

```
SetType ::= SET {ElementTypeList} | SET { }
```

Así, un tipo Set puede incluir cláusulas opcionales, implícitas y de componentes.

Un tipo SET OF («conjunto de») es un número variable de elementos desordenados, todos de un tipo. Una definición de SET OF tiene la siguiente forma:

```
SetOfType ::= SET OF Type | SET
```

La notación SET se tiene que interpretar como SET OF ANY; el tipo ANY se explica en una subsección posterior.

El término **tipo tagged** (tipo marca) es de alguna forma sin nombre ya que todos los tipos de datos en ASN.1 tienen asociado una marca. El estándar ASN.1 define un tipo marca como sigue:

Un tipo definido para referenciar un tipo existente y una marca; el nuevo tipo es isomórfico al tipo existente, pero es una forma distinta. En todos los esquemas de codificación un valor del tipo nuevo se puede distinguir de un valor del tipo antiguo.

La acción de marcar es útil para distinguir tipos dentro de una aplicación. Puede ser deseable tener varios nombres de tipos diferentes, como nombre_empleado y nombre_cliente que son esencialmente del mismo tipo. Para algunas estructuras, es necesario marcar para distinguir tipos de componentes dentro del tipo estructura. Por ejemplo, a los componentes opcionales de un tipo SET o SEQUENCE se les da normalmente marcas específicas de contexto distintas para evitar las ambigüedades.

Existen dos categorías de tipos de marcas: tipos de marcas implícitas y tipos de marcas explícitas. Un tipo de marca implícita se deriva de otro tipo *reemplazando* la marca (nombre de marca antiguo,

número de marca antiguo) de un tipo antiguo con una marca nueva (nombre de marca nueva, número de marca nuevo). Por motivos de codificación sólo se utilizan las marcas nuevas.

Un tipo de marca explícito se deriva de otro tipo *incorporando* una nueva marca al tipo subyacente. En efecto, un tipo de marca explícito es un tipo de estructura con un componente, el tipo subyacente. Por motivos de codificación, tanto la marca nueva como la antigua se deben reflejar en la codificación.

Una marca implícita da lugar a una codificación más corta, pero puede ser necesaria una marca explícita para evitar ambigüedades si la marca del tipo subyacente está indeterminada (por ejemplo, si el tipo subyacente es CHOICE o ANY).

Los tipos **CHOICE** («elección») y **ANY** («cualquiera») son tipos de datos sin marcas. La razón para esto es que cuando se asigna un valor particular al tipo, entonces se debe asignar un tipo particular al mismo tiempo. Así, el tipo se asigna durante la ejecución.

El tipo CHOICE es una lista de tipos alternativos conocidos. Sólo uno de estos tipos será realmente utilizado al crear un valor. Ya se indicó antes que un tipo se puede ver como una colección de valores. El tipo CHOICE es la unión de conjuntos de valores de todos los tipos de componentes indicados en el tipo CHOICE. Este tipo es útil cuando los valores que se van a describir pueden ser de tipos diferentes dependiendo de las circunstancias, y todos los tipos posibles se conocen por adelantado.

La notación para definir el tipo CHOICE es como sigue:

```
ChoiceType ::= CHOICE {AlternativeTypeList}
AlternativeTypeList ::= NamedType | AlternativeTypeList, NamedType
```

El tipo ANY describe un valor arbitrario de un tipo arbitrario. La notación es sencillamente:

```
AnyType ::= ANY
```

Este tipo es útil cuando los valores que se van a describir pueden ser de tipos diferentes pero los tipos posibles no son conocidos por adelantado.

Subtipos

Un subtipo se deriva de un tipo padre restringiendo el conjunto de valores definidos para un tipo padre. Esto es, el conjunto de valores para el subtipo es un subconjunto del conjunto de valores para el tipo padre. El proceso de crear un subtipo se puede ampliar a más de un nivel: esto es, un subtipo puede ser él mismo, el padre e incluso un subtipo más reducido.

En el estándar se proporcionan seis formas diferentes de notación para designar los valores de un subtipo. La Tabla 19.3 muestra cuáles de estas formas se pueden aplicar a tipos particulares de padres. El resto de esta subsección proporciona una descripción general de cada uno de ellos.

Un **subtipo de valor único** es un listado explícito de todos los valores que el subtipo puede tomar. Por ejemplo:

```
Primos menores ::= INTEGER (2 | 3 | 5 | 7 | 11 | 13 | 17 | 19 | 23 | 29)
```

En este caso, Primos-menores es un subtipo del tipo interior INTEGER. Otro ejemplo:

```
Meses ::= ENUMERATED enero (1),
febrero (2),
marzo (3),
abril (4),
mayo (5),
junio (6),
julio (7),
agosto (8),
```

Tabla 19.3. Aplicabilidad de los conjuntos de valores de subtipo.

Tipo (o derivado de tal tipo mediante marcado)	Valor único	Subtipo contenido	Rango de valor	Restricción de tamaño	Alfabeto permitido	Subtipo interno
Boolean	✓	✓				
Integer	✓	✓	✓			
Enumerated	✓	✓				
Real	✓	✓	✓			
Object Identifier	✓	✓				
Bit String	✓	✓		✓		
Octet String	✓	✓		✓		
Character String Types	✓	✓		✓	✓	
Sequence	✓	✓				✓
Sequence Of	✓	✓		✓		✓
Set	✓	✓				✓
Set Of	✓	✓		✓		✓
Any	✓	✓				
Choice	✓	✓				✓

septiembre (9),
octubre (10),
noviembre (11),
diciembre (12)

Primer-cuatrimestre ::= Meses (enero | febrero | marzo)

Segundo-cuatrimestre ::= Meses (abril | mayo | junio)

Primer-cuatrimestre y Segundo-cuatrimestre son ambos subtipos del tipo enumerado Meses.

Un **subtipo contained** (contenido) se utiliza para formar nuevos subtipos a partir de subtipos existentes. El subtipo contenido incluye todos los valores de los subtipos que él contiene. Por ejemplo:

Primera-mitad ::= Meses (INCLUDES Primer-cuatrimestre | INCLUDES Segundo-cuatrimestre)

Un subtipo contenido puede también incluir un listado de valores explícitos.

Primer-trimestre ::= Meses (INCLUDES Primer-cuatrimestre | abril)

Un **subtipo value range** (de rango de valor) se aplica sólo a los tipos INTEGER y REAL. Se especifica dando los valores numéricos de los extremos del rango. Se pueden utilizar los valores especiales PLUS-INFINITY («más infinito») y MINUS INFINITY («menos infinito»). Los valores especiales MIN y MAX se pueden utilizar para indicar los valores permitidos máximos y mínimos del padre. Cada ex-

tremo del rango está abierto o cerrado. Cuando está abierto, la especificación del punto final incluye el símbolo 'menor que' («<»). Las definiciones siguientes son equivalentes:

```
EnteroPositivo ::= INTEGER (0<..PLUS-INFINITY)
EnteroPositivo ::= INTEGER (1..PLUS-INFINITY)
EnteroPositivo ::= INTEGER (0<..MAX)
EnteroPositivo ::= INTEGER (1..MAX)
```

Las siguientes líneas son equivalentes:

```
EnteroNegativo ::= INTEGER (MINUS-INFINITY..<0)
EnteroNegativo ::= INTEGER (MINUS-INFINITY..-1)
EnteroNegativo ::= INTEGER (MIN..

```

La **restricción de alfabeto permitido** sólo se puede aplicar a los tipos de cadenas de caracteres. Un tipo de alfabeto permitido consta de todos los valores (cadenas) que se pueden construir utilizando un sub-alfabeto del tipo padre. Ejemplos:

```
Teclas de Tono ::= IA5String (FROM
  ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" | "*" | "#"))
Cadena de Dígitos ::= IA5String (FROM
  ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"))
```

Una **restricción de tamaño** limita el número de artículos en un tipo. Sólo se puede aplicar a tipos de cadena (cadena de bit, cadena de octetos, cadena de caracteres) y a tipos SEQUENCE OF y SET OF. Los artículos que están restringidos dependen del tipo padre como sigue:

Tipo	Unidad de medida
cadena de bit	bit
cadena de octetos	octeto
cadena de caracteres	carácter
Sequence-of	valor de componente
Set-of	valor de componente

Como ejemplo de tipo de cadena numérica, la Recomendación X.121 especifica que los números internacionales, que se utilizan para direccionamiento de sistemas finales en redes de datos públicas, incluyendo las redes X.25, deben de constar de al menos 5 dígitos pero no más de 14 dígitos. Esto se podría especificar como sigue:

```
DatoNumérico—itl ::= DigitString (SIZE (5..14))
```

Ahora considere una lista de parámetros para un mensaje que puede incluir hasta doce parámetros:

```
Lista-de-parámetros ::= SET SIZE (0..12) OF Parámetros
```

Una **restricción de tipo interno** se puede aplicar a los tipos SEQUENCE, SEQUENCE OF, SET, SET OF y CHOICE. Un subtipo interno incluye en su conjunto de valores sólo aquellos valores del tipo padre que satisfacen una o más restricciones y/o valores de los componentes del tipo padre. Éste es un subtipo más bien complejo y sólo se dan aquí unos pocos ejemplos.

Considérese una unidad de datos de protocolo (PDU) que puede tener cuatro campos diferentes, sin ningún orden particular:

```
PDU ::= SET alpha [0] INTEGER,
  beta [1] IA5String OPTIONAL
```

```
gamma [2] SEQUENCE OF Parámetros,
delta [3] BOOLEAN
```

La especificación de un test que requiere que Boolean sea falso y el Integer sea negativo puede realizarse así:

```
TestPDU ::= PDU (WITH COMPONENTS {..., delta (FALSE), alpha
(MIN.. <0)})
```

Para especificar además que el parámetro beta esté presente y con una longitud de 5 o 12 caracteres se utiliza la siguiente línea:

```
FurtherTestPDU ::= TestPDU (WITH COMPONENTS {..., beta (SIZE
(5 | 12) PRESENT)})
```

Como otro ejemplo, considere el uso de un subtipo interior en una construcción SEQUENCE OF:

```
Bloque-texto ::= SEQUENCE OF VisibleString
Dirección ::= Bloque-texto (SIZE (1..6) | WITH COMPONENTS (SIZE (1..32)))
```

Este texto indica que la dirección consta de 1 a 6 bloques de texto, y que cada bloque de texto tiene una longitud de 1 a 32 caracteres.

Ejemplo de PDU

Como ejemplo, considere la especificación ASN.1 del formato de las unidades de datos de protocolo para el protocolo SNMPv2 (descrito más tarde en este capítulo). La especificación del estándar se reproduce en la Figura 19.3.

```
SNMPv2-PDU DEFINITIONS ::= BEGIN

PDUs ::= CHOICE { obtener-petición           GetRequest-PDU
                  obtener-siguiente-petición GetNextRequest-PDU
                  obtener-grupo-petición    GetBulkRequest-PDU
                  respuesta                   Response-PDU
                  establecer-petición         SetRequest-PDU
                  informar-petición           InformRequest-PDU
                  snmp V2-trap                SNMPv2-Trap-PDU
                  informe                     Report-PDU
                  }

--PDUs

GetRequest-PDU      ::= [0] IMPLICIT PDU
GetNextRequest-PDU  ::= [1] IMPLICIT PDU
Response-PDU        ::= [2] IMPLICIT PDU
SetRequest-PDU       ::= [3] IMPLICIT PDU
GetBulkRequest-PDU   ::= [5] IMPLICIT BulkPDU
InformRequest-PDU    ::= [6] IMPLICIT PDU
SNMPv2-Trap-PDU     ::= [7] IMPLICIT PDU
```

```
max-variable_colección INTEGER ::= 2147483647
```

Figura 19.3. Definiciones de formato de PDU de SNMPv2 (página 1 de 2).


```

PDU ::= SEQUENCE {
    identificador-solicitud Integer32,
    categoria-error          INTEGER {
                                noError (0),           --ignorado a veces
                                tooBig (1),
                                noSuchName (2),        --para compatibilidad de representante
                                badValue (3),          --para compatibilidad de representante
                                readOnly (4),          --para compatibilidad de representante
                                genError (5),
                                noAccess (6),
                                wrongType (7),
                                wrongLength (8),
                                wrongEncoding (9),
                                wrongValue (10),
                                noCreation (11),
                                inconsistentValue (12),
                                resourceUnavailable (13),
                                commitFailed (14),
                                undoFailed (15),
                                authorizationError (16),
                                notWritable (17),
                                inconsistentName (18) },
    indice-error             INTEGER (0..max-variable-colección), --ignorado a veces
    variable-colección       VarBindList }                    --los valores son a veces
                                                                --ignorados

BulkPDU ::= SEQUENCE {
    identificador-solicitud Integer32,
    no repetidor             INTEGER (0..max-variable-colección),
    repeticiones = max.      INTEGER (0..max-variable-colección),
    variable-colección       VarBindList }                    --los valores son ignorados

--variable colección

VarBind ::= SEQUENCE {
    nombre ObjectName,
    CHOICE {
        valor          ObjectSyntax,
        unspecified     NULL, --en solicitudes de recuperación
                                --excepciones en respuestas:
        noSuchObject [0]      IMPLICIT NULL,
        noSuchInstance [1]    IMPLICIT NULL,
        endOfMibView [2]      IMPLICIT NULL } }

--lista variable-colección

VarBindList ::= SEQUENCE (SIZE (0..max-variable-colección)) OF VarBind

END

```

Figura 19.3. Definiciones de formato de PDU de SNMPv2 (página 2 de 2).

La construcción del nivel más alto utiliza el tipo CHOICE para describir una variable seleccionada de una colección. Así, cualquier ejemplar del tipo PDU será uno de los ocho tipos alternativos. Obsérvese que cada una de las elecciones se etiqueta con un nombre. Todos las PDU definidas de esta forma tienen el mismo formato pero diferentes etiquetas, con la excepción de la PDU GetBulkRequest. El formato consta de una secuencia de cuatro elementos. El segundo elemento, es todo de error, enumera 19 valores enteros posibles, cada uno con una etiqueta. En el último elemento, se define la variable co-

lección (*binding*) con sintaxis VarBindList, que se define más adelante en el mismo conjunto de definiciones.

La definición BulkPDU es también una secuencia de cuatro elementos, pero difieren de las otras PDU.

VarBindList está definida como una construcción SEQUENCE OF consistente en algún número de elementos de sintaxis VarBind, con una restricción de tamaño de hasta 2147483647, o $2^{31} - 1$, elementos. Cada elemento a su vez es una secuencia de dos valores; el primero es un nombre y el segundo es una elección entre cinco elementos.

19.2. GESTIÓN DE RED—SNMP

Las redes y los sistemas de procesamiento distribuido son de una importancia crítica y creciente en los negocios, gobierno y otras instituciones. Dentro de una institución, la tendencia es hacia redes más grandes, más complejas y dando soporte a más aplicaciones y a más usuarios. A medida que estas redes crecen en escala, existen dos hechos que se hacen penosamente evidentes:

- La red y sus recursos asociados y las aplicaciones distribuidas llegan a ser indispensables para la organización.
- Hay más cosas que pueden ir mal, inutilizar la red o una parte de ella o degradar las prestaciones a un nivel inaceptable.

Una red grande no se puede instalar y gestionar sólo con el esfuerzo humano. La complejidad de un sistema tal impone el uso de herramientas automáticas de gestión de red. La urgencia de la necesidad de esas herramientas se incrementa, y también está en auge la dificultad de suministrar dichas herramientas, si la red incluye equipos de múltiples distribuidores. En respuesta, se han desarrollado normalizaciones para tratar la gestión de red, y que cubren los servicios, los protocolos y la base de información de gestión.

Esta sección comienza con una introducción a los conceptos globales de gestión de red normalizada. El resto de la sección se dedica a la discusión de SNMP, el estándar de gestión de red más utilizado.

SISTEMAS DE GESTIÓN DE RED

Un sistema de gestión de red es una colección de herramientas para monitorizar y controlar la red y que está integrado en los siguientes sentidos:

- Una interfaz de operador sencilla con un conjunto de órdenes potentes, pero agradables para el usuario, para llevar cabo la mayoría o todas las tareas de gestión de red.
- Una cantidad mínima de equipo separado del sistema de gestión. Esto es, la mayor parte del hardware y el software requeridos para la gestión de red están incorporados en el equipo del usuario.

Un sistema de gestión de red consta de hardware extra y software adicional implementados entre los componentes de red existentes. El software se utiliza para efectuar las tareas de gestión de red que residen en los computadores y en los procesadores de comunicaciones (por ejemplo, procesadores frontales y controladores de grupos de terminales). Un sistema de gestión de red está diseñado para ver la red entera como una arquitectura unificada, con direcciones y etiquetas asignadas a cada punto y los atributos específicos de cada elemento y enlace del sistema conocidos. Los elementos activos de la red proporcionan una realimentación regular de información de estado al centro de control de red.

Un sistema de gestión de red tiene los siguientes elementos clave:

- Estación de gestión o gestor.

- Agente.
- Base de información de gestión.
- Protocolo de gestión de red.

La estación de gestión es normalmente un dispositivo autónomo pero puede ser implementado en un sistema compartido. En cualquier caso, la estación de gestión sirve como interfaz entre el gestor de red humano y el sistema de gestión de red. La estación de gestión, tendrá, como mínimo:

- Un conjunto de aplicaciones de gestión para el análisis de los datos, recuperación de fallos, etc.
- Una interfaz a través de la cual el gestor de red puede monitorizar y controlar la red.
- La capacidad de trasladar los requisitos del gestor de red a la monitorización y control real de los elementos en la red.
- Una base de datos de información de gestión de red extraída de las bases de datos de todas las entidades gestionadas en la red.

El otro elemento activo en un sistema de gestión de red es el **agente**. Las plataformas claves, como los computadores, puentes, dispositivos de encaminamiento y concentradores se pueden equipar con software de agente para que puedan ser gestionados desde la estación de gestión. El agente responde a las solicitudes de información desde una estación de gestión, responde a las solicitudes de acción desde la estación de gestión y puede, de una forma asíncrona, proporcionar a la estación de gestión información importante y no solicitada.

El medio por el cual se pueden gestionar los recursos de una red es representando estos recursos como objetos. Cada objeto es, esencialmente, una variable de datos que representa un aspecto del agente de gestión. La colección de objetos se conoce como **base de información de gestión** (MIB, Management Information Base). La MIB funciona como una colección de puntos de accesos al agente por parte de la estación de gestión. Estos objetos están normalizados a través de los sistemas de una clase particular (por ejemplo, todos los puentes contienen los mismos objetos de gestión). La estación de gestión lleva a cabo la función de monitorización mediante el acceso a los valores de los objetos MIB. Una estación de gestión puede causar que una acción tenga efecto en un agente o puede cambiar la configuración de un agente mediante la modificación de los valores de variables específicas.

La estación de gestión y el agente están enlazados por el **protocolo de gestión de red**. El protocolo utilizado para la gestión en redes TCP/IP es el protocolo sencillo de gestión de red (SNMP). Para las redes basadas en OSI, se está desarrollando el protocolo de información de gestión común (CMIP, Common Management Information Protocol). Una versión mejorada de SNMP, conocida como SNMPv2, está proyectada para ambos tipos de redes, basadas en OSI y basadas en TCP/IP. Cada uno de estos protocolos incluye las siguientes capacidades clave:

- **Get:** permite a la estación de gestión obtener del agente los valores de objetos.
- **Set:** permite a la estación de gestión establecer valores de objetos del agente.
- **Notify:** permite a un agente notificar a una estación de gestión la producción de eventos significativos.

En un esquema tradicional de gestión de red centralizado, un computador en la configuración tiene el papel de estación de gestión; puede haber posiblemente una o dos estaciones de gestión con una misión de respaldo. El resto de los dispositivos en la red contiene software de agente y una MIB para permitir la monitorización y control por parte de la estación de gestión. Conforme las redes crecen en tamaño y en carga de tráfico, un sistema centralizado como el indicado no es práctico. Hay demasiada carga situada en la estación de gestión, y hay también mucho tráfico, con informes desde cada agente atravesando la red entera hasta llegar al «cuartel general». En tales circunstancias, una técnica descentralizada y distribuida funciona de mejor forma (por ejemplo, Figura 19.4). En un esquema descentraliza-

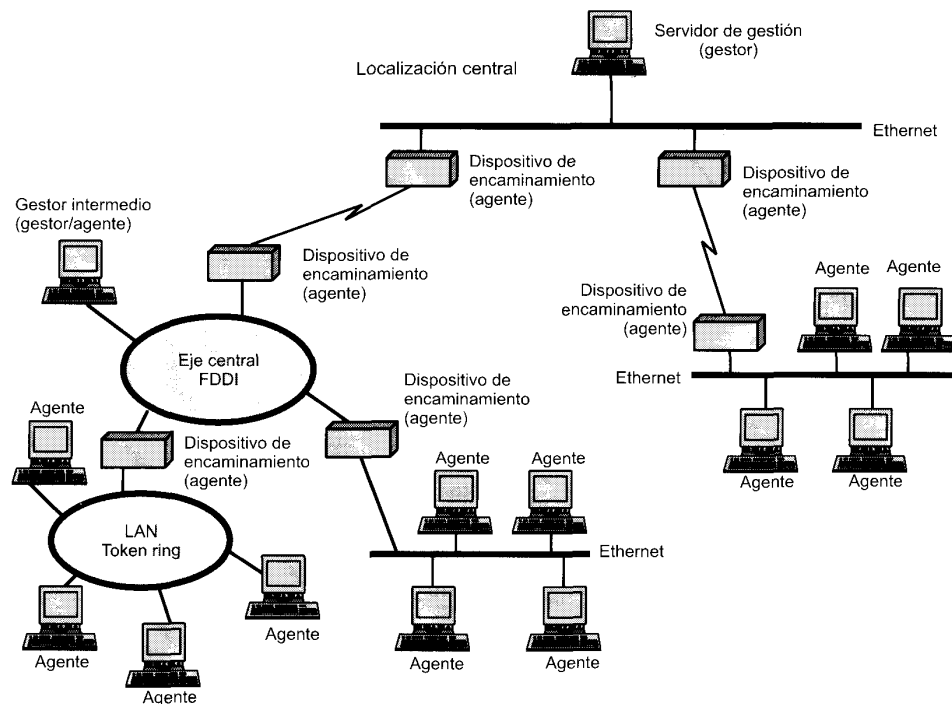


Figura 19.4. Ejemplo de configuración de gestión de una red distribuida.

do de gestión de red, puede haber múltiples estaciones de gestión del nivel más alto, que se podrían denominar servidores de gestión. Cada uno de estos servidores podría gestionar directamente una parte del conjunto total de agentes. Sin embargo, para muchos de estos agentes, el servidor de gestión delega la responsabilidad a un gestor intermedio. El gestor intermedio juega el papel de un gestor para monitorizar y controlar los agentes bajo su responsabilidad. También juega el papel de agente para proporcionar información y aceptar control desde un servidor de gestión de un nivel más alto. Este tipo de arquitectura dispersa la carga de procesamiento y reduce al tráfico total de la red.

PROTOCOLO SIMPLE DE GESTIÓN DE RED VERSIÓN 2 (SNMPv2)

En agosto de 1988, se publicó la especificación SNMP y rápidamente se convirtió en el estándar de gestión de red dominante. Una serie de suministradores ofrecen estaciones de trabajo de gestión de red basadas en SNMP y la mayoría de los vendedores de puentes, dispositivos de encaminamiento, estaciones de trabajo y PC ofrecen paquetes de agente SNMP que permiten que sus productos sean gestionados por una estación de gestión SNMP.

Como el nombre sugiere, SNMP es una herramienta sencilla para la gestión de red. Define una base de información de gestión (MIB) limitada y fácil de implementar de variables escalares y tablas de dos dimensiones, y define un protocolo para permitir a un gestor obtener y establecer variables MIB y para permitir a un agente emitir notificaciones no solicitadas, llamadas intercepciones (*traps*). Esta simplicidad es la potencia de SNMP. SNMP se implementa de una forma fácil y consume un tiempo modesto del procesador y de recursos de red. También, la estructura del protocolo y de la MIB es suficientemente directa de forma que no es difícil alcanzar la interacción entre estaciones de gestión y software de agente de varios vendedores.